

CPP102: PROGRAMMING LOGIC AND DESIGN

Week 5 Lecture and Laboratory Module

Conditional Statements in Python

Department of Computer Engineering

PART A. LECTURE MODULE

1. Week Number and Topic

Week: 5

Course Title: CPP102 — Programming Logic and Design (2 units, Lecture-Laboratory)

Topic: Conditional Statements in Python

Subtopics: decision-making in programming, Boolean conditions, comparison operators, logical operators, Python indentation, the simple if statement, the if-else statement, the if-elif-else statement, nested if statements, combining multiple conditions, common conditional-statement errors, flowcharts for decision structures, and developing algorithms, pseudocode, flowcharts, and Python programs that use selection.

2. Intended Learning Outcomes

At the end of the module, students should be able to:

1. Evaluate comparison and logical expressions used in Python conditional statements.
2. Construct algorithms, pseudocode, and flowcharts that use selection or decision structures.
3. Develop and test Python programs using if, if-else, if-elif-else, and nested conditional statements.

3. Week 5 Overview

In the previous weeks, every program you wrote executed its statements from top to bottom, one after another, without skipping anything. Real programs, however, must often make decisions. A conditional statement allows a program to choose which statements to execute based on whether a condition evaluates to True or False. This ability to select a path of execution is called the selection or decision control structure, and it is one of the three basic control structures of programming, together with sequence and repetition.

Decision-making is everywhere in daily life, and the rules we follow can be written as conditions:

- A student passes a course if the final grade is at least 75.
- A customer receives a discount if the total purchase reaches the required amount.
- A person may enter a venue if the age requirement is satisfied.

This module builds directly on the Week 4 topics: variables, data types, type conversion, arithmetic operators, comparison operators, logical operators, and user input and program output. In Week 4, you learned how to form expressions such as `grade >= 75`; this week, you will use those expressions to control what your program does.

4. Review of Boolean Expressions

A condition is an expression that produces one of only two possible values: True or False. These are called Boolean values, named after the mathematician George Boole. Conditions are usually formed using comparison operators, which compare two values.

Operator	Meaning	Example
==	Equal to	score == 75
!=	Not equal to	age != 18
>	Greater than	temperature > 30
<	Less than	quantity < 5
>=	Greater than or equal to	grade >= 75
<=	Less than or equal to	balance <= 0

A comparison can be printed directly, and Python will display the Boolean result. Study the following example:

```
grade = 82
print(grade >= 75)
print(grade < 75)
```

Output:

True
False

Since 82 is greater than or equal to 75, the first expression is True. The second expression asks whether 82 is less than 75, which is False. Here are more evaluated examples:

Expression	Result	Reason
10 == 10	True	Both values are equal
7 != 7	False	The values are equal, so “not equal” is false
15 > 20	False	15 is not greater than 20
75 >= 75	True	75 is equal to 75, which satisfies “or equal to”
0 <= 3	True	0 is less than 3

Reminder: a single equal sign (=) assigns a value to a variable, while a double equal sign (==) compares two values. Confusing the two is one of the most common beginner errors.

5. Logical Operators

Logical operators combine or modify Boolean values. They allow a program to check several conditions at the same time.

Operator	Description
and	Both conditions must be true for the result to be True
or	At least one condition must be true for the result to be True
not	Reverses the Boolean result

Truth table for and:

A	B	A and B
True	True	True
True	False	False
False	True	False
False	False	False

Truth table for or:

A	B	A or B
True	True	True
True	False	True
False	True	True
False	False	False

Truth table for not:

A	not A
True	False
False	True

Study the following beginner-friendly example:

```

age = 20
has_id = True
print(age >= 18 and has_id)

```

Output:

```

True

```

The expression `age >= 18` evaluates to `True` because 20 is at least 18. The variable `has_id` already holds the value `True`. Since `and` requires both sides to be true, and both sides are indeed true, the final result is `True`. If either side were `False`, the whole expression would become `False`.

Checkpoint A: Evaluate the Condition

Write True or False for each expression. Answers are in the instructor’s answer key.

No.	Expression	True or False?
1	<code>10 > 5</code>	
2	<code>8 == 8</code>	
3	<code>12 != 12</code>	
4	<code>75 >= 75</code>	
5	<code>20 < 10 or 5 == 5</code>	
6	<code>not (7 > 3)</code>	
7	<code>15 <= 14</code>	
8	<code>3 > 1 and 2 > 5</code>	
9	<code>not (4 == 4) and 10 > 2</code>	
10	<code>6 != 7 or 8 < 3</code>	

6. Python Indentation

Python uses indentation (spaces at the beginning of a line) to identify which statements belong to a conditional block. Statements indented under an `if` line are executed only when the condition

is True. This is different from many other languages that use braces or keywords; in Python, indentation is part of the grammar of the language, not just a matter of style.

Correct example:

```
grade = 80
if grade >= 75:
    print("Passed")
```

Incorrect example:

```
grade = 80
if grade >= 75:
print("Passed")
```

The second example causes an IndentationError. After the colon at the end of the if line, Python expects at least one indented statement that forms the body of the condition. Because print("Passed") starts at the left margin, Python cannot tell that it belongs to the if statement, so the program does not run.

Reminder: use four spaces for every indentation level. Do not randomly mix tabs and spaces, because Python treats them differently and the program may refuse to run.

7. The Simple if Statement

The simple if statement executes a block of statements only when the condition is True. When the condition is False, the block is skipped and the program continues with the next unindented statement.

Syntax:

```
if condition:
    statement
```

A simple if statement is appropriate when something should happen only in one situation, and nothing special needs to happen otherwise. For example, printing a congratulatory message only when a student passes.

Example 5.1: Checking Whether a Number Is Positive

Problem Statement: Ask the user for a number. If the number is greater than zero, display “The number is positive.”

Input: a number entered by the user.

Process: compare the number with zero using the > operator.

Output: the message “The number is positive.” (only when the condition is True).

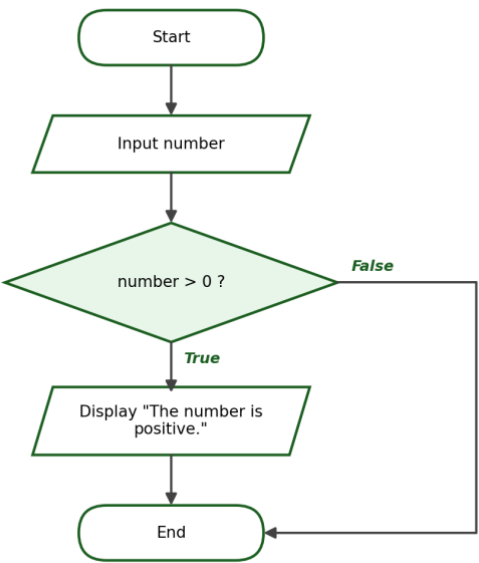
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter a number. Store it as number.
- Step 3: If number is greater than 0, display “The number is positive.”
- Step 4: End.

Pseudocode:

```
START
INPUT number
IF number > 0 THEN
    DISPLAY "The number is positive."
END IF
END
```

Flowchart:



Python Program:

```
number = float(input("Enter a number: "))
if number > 0:
    print("The number is positive.")
```

Sample Run:

```
Enter a number: 7
The number is positive.
```

Explanation: The input 7 is converted to the float 7.0. Since 7.0 > 0 is True, the indented print statement executes. If the user enters -3, the condition is False, the block is skipped, and the program simply ends without displaying anything.

Example 5.2: Checking Whether a Student Passed

Problem Statement: Ask the user for a grade. If the grade is at least 75, display “Congratulations, you passed!”

Input: the student’s grade.

Process: compare the grade with the passing mark 75 using >=.

Output: a congratulatory message (only when the grade is 75 or higher).

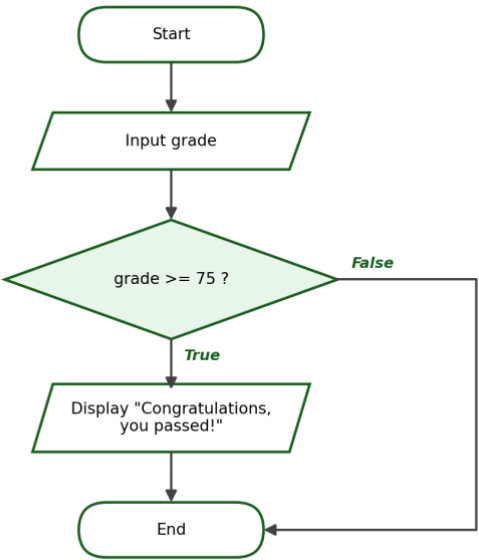
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the grade. Store it as grade.
- Step 3: If grade is greater than or equal to 75, display “Congratulations, you passed!”
- Step 4: End.

Pseudocode:

```
START
INPUT grade
IF grade >= 75 THEN
    DISPLAY "Congratulations, you passed!"
END IF
END
```

Flowchart:



Python Program:

```
grade = float(input("Enter your grade: "))
if grade >= 75:
    print("Congratulations, you passed!")
```

Sample Run:

```
Enter your grade: 85
Congratulations, you passed!
```

Explanation: The condition grade >= 75 is True for the input 85, so the message is displayed. Note that the boundary value 75 also satisfies the condition because of the “or equal to” part of the operator.

Example 5.3: Checking Whether a Purchase Qualifies for a Discount

Problem Statement: Ask the user for the total purchase amount. If the amount is at least ₱1,000, display “You qualify for a discount.”

Input: the total purchase amount.

Process: compare the purchase with 1000 using >=.

Output: a qualification message (only when the purchase is ₱1,000 or more).

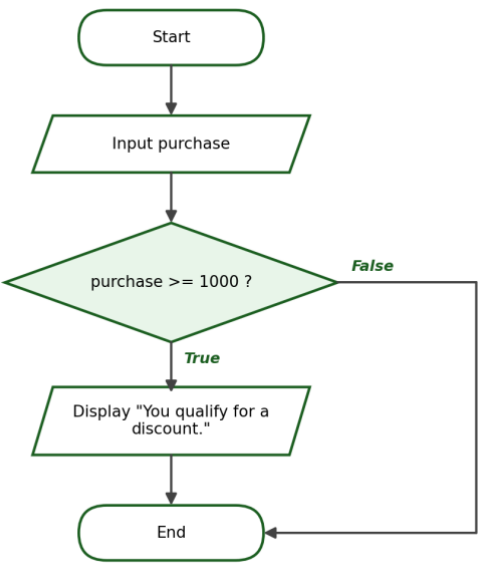
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the total purchase. Store it as purchase.
- Step 3: If purchase is greater than or equal to 1000, display “You qualify for a discount.”
- Step 4: End.

Pseudocode:

```
START
INPUT purchase
IF purchase >= 1000 THEN
    DISPLAY "You qualify for a discount."
END IF
END
```

Flowchart:



Python Program:

```
purchase = float(input("Enter total purchase: "))
if purchase >= 1000:
    print("You qualify for a discount.")
```

Sample Run:

```
Enter total purchase: 1500
You qualify for a discount.
```

Explanation: Since 1500 >= 1000 is True, the message appears. For an input of 800 the condition is False and nothing is displayed. A simple if is enough here because the problem does not require any message for customers who do not qualify.

8. The if-else Statement

The if-else statement provides two paths: one block runs when the condition is True, and a different block runs when the condition is False. Exactly one of the two blocks always executes.

Syntax:

```
if condition:
    statement_if_true
else:
    statement_if_false
```

Example 6.1: Pass or Fail

Problem Statement: Ask the user for a grade. Display “Passed” if the grade is at least 75; otherwise, display “Failed”.

Input: the student’s grade.

Process: compare the grade with 75.

Output: “Passed” or “Failed”.

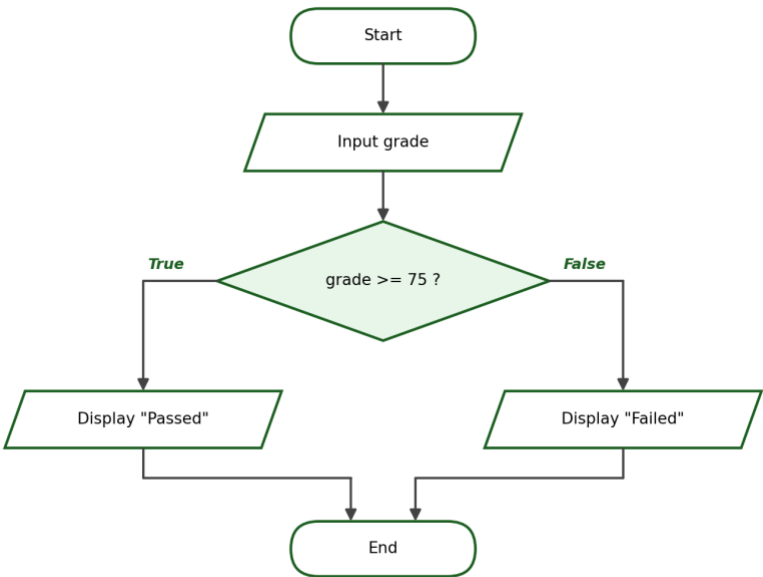
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the grade. Store it as grade.
- Step 3: If grade is greater than or equal to 75, display “Passed”.
- Step 4: Otherwise, display “Failed”.
- Step 5: End.

Pseudocode:

```
START
INPUT grade
IF grade >= 75 THEN
    DISPLAY "Passed"
ELSE
    DISPLAY "Failed"
END IF
END
```

Flowchart:



Python Program:

```
grade = float(input("Enter your grade: "))
if grade >= 75:
    print("Passed")
else:
    print("Failed")
```

Sample Run 1:

```
Enter your grade: 88
Passed
```

Sample Run 2:

```
Enter your grade: 74
Failed
```

Explanation: Unlike the simple if in Example 5.2, this program always produces an answer. A grade of 88 satisfies the condition, so the True block runs; a grade of 74 does not, so the else block runs instead.

Example 6.2: Even or Odd Number

Problem Statement: Ask the user for an integer and determine whether it is even or odd.

Input: an integer.

Process: compute the remainder of the number divided by 2 using the modulo operator (%).

Output: “Even number” or “Odd number”.

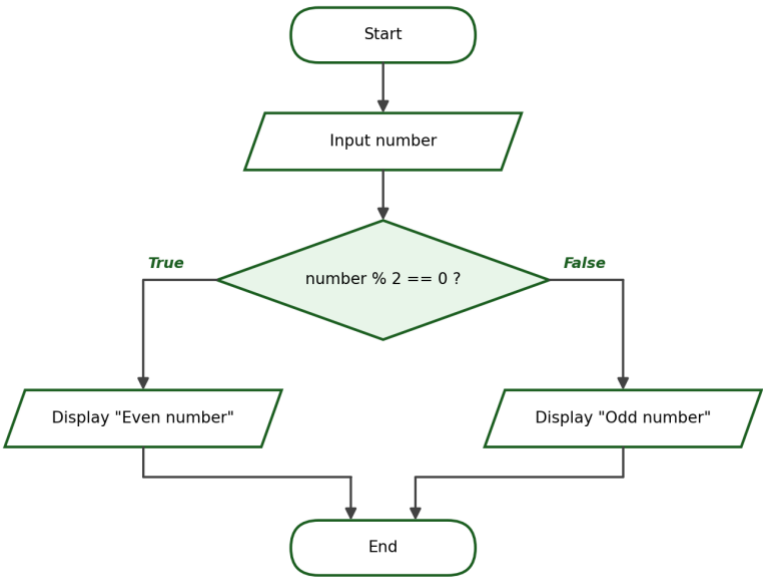
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter a number. Store it as number.
- Step 3: Compute the remainder of number divided by 2.
- Step 4: If the remainder equals 0, display “Even number”.
- Step 5: Otherwise, display “Odd number”.
- Step 6: End.

Pseudocode:

```
START
INPUT number
IF number MOD 2 == 0 THEN
    DISPLAY "Even number"
ELSE
    DISPLAY "Odd number"
END IF
END
```

Flowchart:



Python Program:

```
number = int(input("Enter a number: "))
if number % 2 == 0:
    print("Even number")
else:
    print("Odd number")
```

Sample Run 1:

```
Enter a number: 12
Even number
```

Sample Run 2:

```
Enter a number: 7
Odd number
```

Explanation: An even number leaves no remainder when divided by 2, so number % 2 == 0 is True. The number 12 gives remainder 0 (even), while 7 gives remainder 1 (odd).

Example 6.3: Adult or Minor

Problem Statement: Ask the user for an age. Display “Adult” if the age is 18 or above; otherwise, display “Minor”.

- Input:** the person’s age.
- Process:** compare the age with 18.
- Output:** “Adult” or “Minor”.

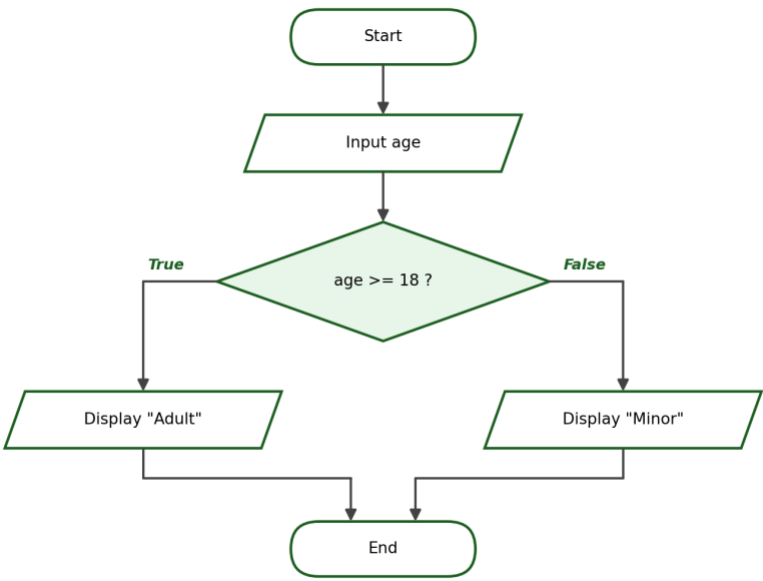
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the age. Store it as age.
- Step 3: If age is greater than or equal to 18, display “Adult”.
- Step 4: Otherwise, display “Minor”.
- Step 5: End.

Pseudocode:

```
START
INPUT age
IF age >= 18 THEN
    DISPLAY "Adult"
ELSE
    DISPLAY "Minor"
END IF
END
```

Flowchart:



Python Program:

```
age = int(input("Enter your age: "))
if age >= 18:
    print("Adult")
else:
    print("Minor")
```

Sample Run 1:

```
Enter your age: 21
Adult
```

Sample Run 2:

```
Enter your age: 15
Minor
```

Explanation: The age is read as an integer because ages are counted in whole years. The boundary value 18 is classified as “Adult” because the operator is >= and not >.

Example 6.4: Sufficient or Insufficient Balance

Problem Statement: Ask the user for an account balance and a withdrawal amount. If the withdrawal does not exceed the balance, deduct it and display the remaining balance; otherwise, display “Insufficient balance”.

Input: the account balance and the withdrawal amount.

Process: compare the withdrawal with the balance; if allowed, subtract the withdrawal from the balance.

Output: the remaining balance, or the message “Insufficient balance”.

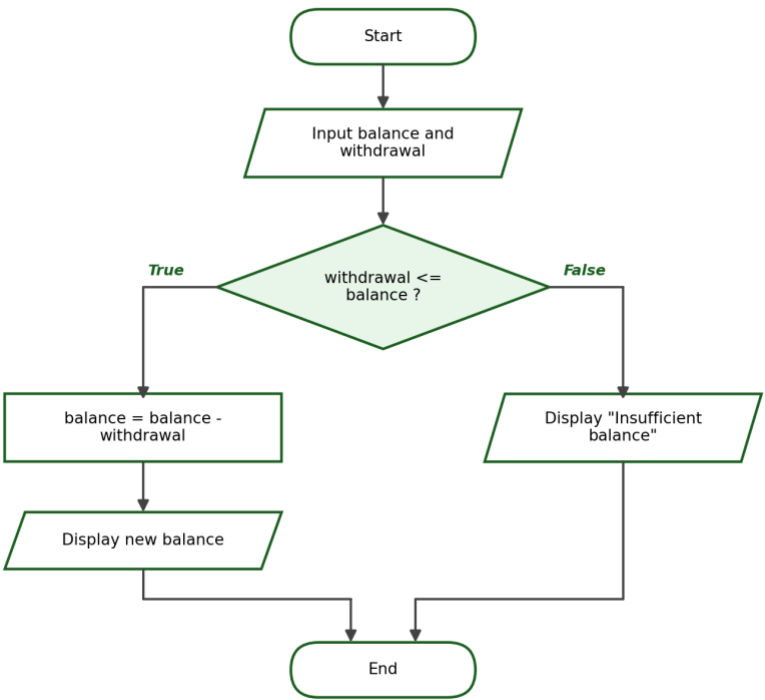
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the balance. Store it as balance.
- Step 3: Ask the user to enter the withdrawal amount. Store it as withdrawal.
- Step 4: If withdrawal is less than or equal to balance, compute balance = balance - withdrawal and display the remaining balance.
- Step 5: Otherwise, display “Insufficient balance”.
- Step 6: End.

Pseudocode:

```
START
INPUT balance
INPUT withdrawal
IF withdrawal <= balance THEN
    balance = balance - withdrawal
    DISPLAY "Remaining balance: " + balance
ELSE
    DISPLAY "Insufficient balance"
END IF
END
```

Flowchart:



Python Program:

```
balance = float(input("Enter account balance: "))
withdrawal = float(input("Enter withdrawal amount: "))
if withdrawal <= balance:
    balance = balance - withdrawal
    print(f"Remaining balance: ₱{balance:.2f}")
else:
    print("Insufficient balance")
```

Sample Run 1:

```
Enter account balance: 5000
Enter withdrawal amount: 1500
Remaining balance: ₱3500.00
```

Sample Run 2:

```
Enter account balance: 1000
Enter withdrawal amount: 2500
Insufficient balance
```

Explanation: This example shows that the True block may contain more than one statement: a computation followed by an output. The formatted output `{balance:.2f}` displays the amount with exactly two decimal places.

9. The if-elif-else Statement

The if-elif-else statement is used when a program must select exactly one result from several possible conditions. Python checks the conditions from top to bottom and executes the block of the first condition that is True. If none of the conditions is True, the else block executes. The keyword elif is short for “else if”.

Syntax:

```
if condition_1:
    statement_1
elif condition_2:
    statement_2
else:
    default_statement
```

The grade classification used in this module follows the ranges below. Notice that the ranges do not overlap and have no gaps:

Grade	Classification
90–100	Excellent
85–89	Very Good
80–84	Good
75–79	Passed
Below 75	Failed

Example 7.1: Grade Classification

Problem Statement: Ask the user for a grade and display its classification using the table above.

Input: the student’s grade.

Process: compare the grade against the boundaries 90, 85, 80, and 75, from highest to lowest.

Output: one of: “Excellent”, “Very Good”, “Good”, “Passed”, or “Failed”.

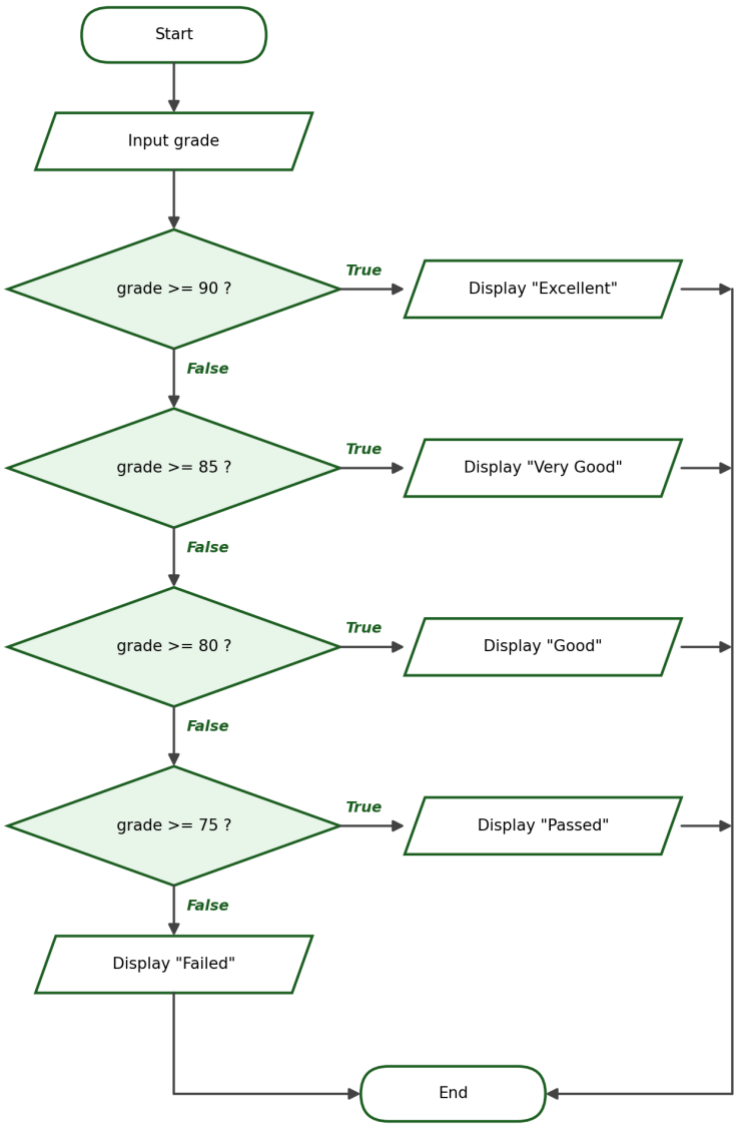
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the grade. Store it as grade.
- Step 3: If grade >= 90, display “Excellent”.
- Step 4: Otherwise, if grade >= 85, display “Very Good”.
- Step 5: Otherwise, if grade >= 80, display “Good”.
- Step 6: Otherwise, if grade >= 75, display “Passed”.
- Step 7: Otherwise, display “Failed”.
- Step 8: End.

Pseudocode:

```
START
INPUT grade
IF grade >= 90 THEN
    DISPLAY "Excellent"
ELSE IF grade >= 85 THEN
    DISPLAY "Very Good"
ELSE IF grade >= 80 THEN
    DISPLAY "Good"
ELSE IF grade >= 75 THEN
    DISPLAY "Passed"
ELSE
    DISPLAY "Failed"
END IF
END
```

Flowchart:



Python Program:

```
grade = float(input("Enter your grade: "))
if grade >= 90:
    print("Excellent")
elif grade >= 85:
    print("Very Good")
elif grade >= 80:
    print("Good")
elif grade >= 75:
    print("Passed")
else:
    print("Failed")
```

Sample Run 1:

```
Enter your grade: 95
Excellent
```

Sample Run 2:

```
Enter your grade: 87
Very Good
```

Sample Run 3:

```
Enter your grade: 75
Passed
```

Explanation: The conditions are arranged from the highest boundary to the lowest. A grade of 87 fails the first check (87 >= 90 is False) but passes the second (87 >= 85 is True), so “Very Good” is displayed and the remaining conditions are skipped.

Example 7.2: Temperature Classification

Problem Statement: Ask the user for the temperature in degrees Celsius and classify it: 35 and above is “Hot”, 25 to below 35 is “Warm”, 15 to below 25 is “Cool”, and below 15 is “Cold”.

Input: the temperature in Celsius.

Process: compare the temperature against the boundaries 35, 25, and 15, from highest to lowest.

Output: one of: “Hot”, “Warm”, “Cool”, or “Cold”.

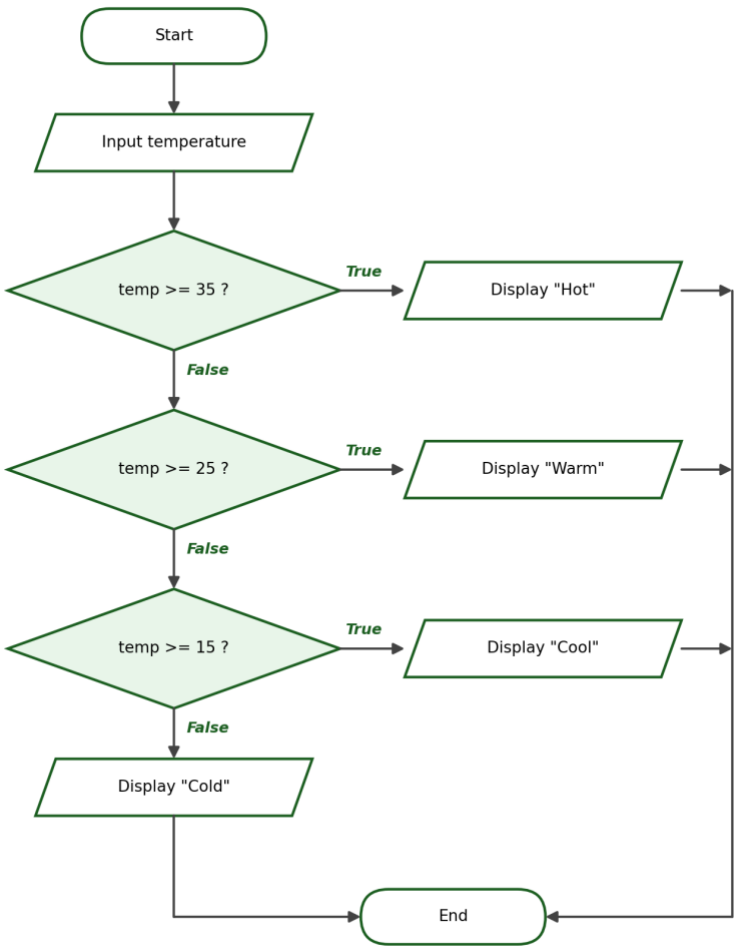
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the temperature. Store it as temperature.
- Step 3: If temperature >= 35, display “Hot”.
- Step 4: Otherwise, if temperature >= 25, display “Warm”.
- Step 5: Otherwise, if temperature >= 15, display “Cool”.
- Step 6: Otherwise, display “Cold”.
- Step 7: End.

Pseudocode:

```
START
INPUT temperature
IF temperature >= 35 THEN
  DISPLAY "Hot"
ELSE IF temperature >= 25 THEN
  DISPLAY "Warm"
ELSE IF temperature >= 15 THEN
  DISPLAY "Cool"
ELSE
  DISPLAY "Cold"
END IF
END
```

Flowchart:



Python Program:

```
temperature = float(input("Enter the temperature in Celsius: "))
if temperature >= 35:
    print("Hot")
elif temperature >= 25:
    print("Warm")
elif temperature >= 15:
    print("Cool")
else:
    print("Cold")
```

Sample Run 1:

```
Enter the temperature in Celsius: 36
Hot
```

Sample Run 2:

Enter the temperature in Celsius: 28
Warm

Sample Run 3:

Enter the temperature in Celsius: 10
Cold

Explanation: Each elif is checked only when all the conditions above it are False. This is why the second condition does not need to say `25 <= temperature < 35` — reaching it already guarantees that the temperature is below 35.

Example 7.3: Customer Membership Level

Problem Statement: A store classifies customers by their total purchases for the year: ₱10,000 and above is Platinum, ₱5,000 to below ₱10,000 is Gold, ₱1,000 to below ₱5,000 is Silver, and below ₱1,000 is Regular. Ask for the total purchases and display the membership level.

Input: the customer’s total purchases for the year.

Process: compare the total against the boundaries 10000, 5000, and 1000, from highest to lowest.

Output: the membership level.

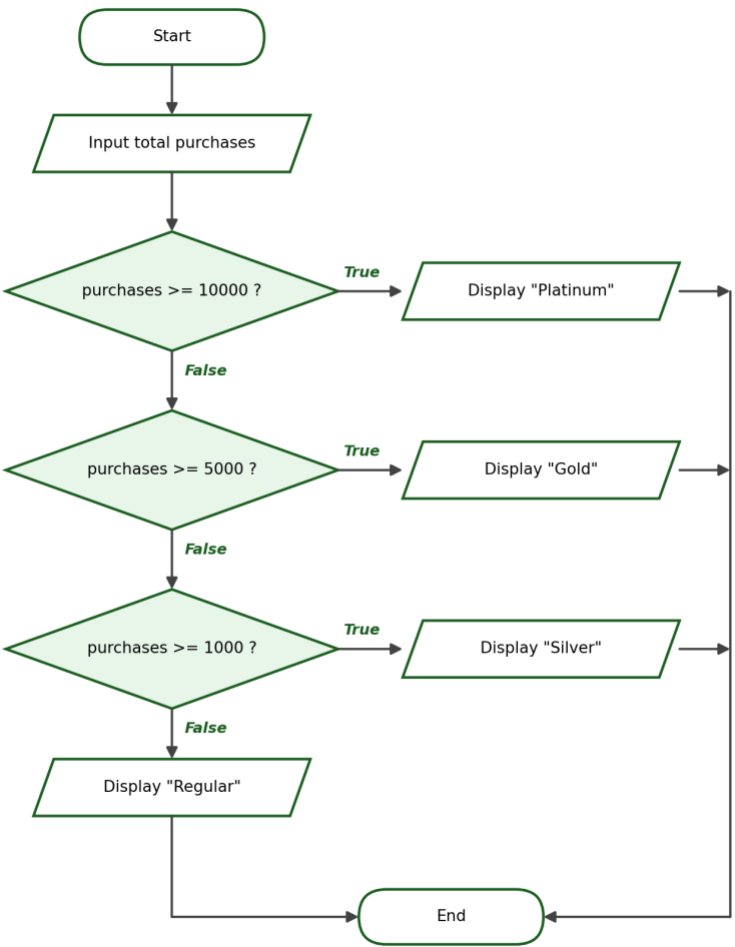
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the total purchases. Store it as purchases.
- Step 3: If purchases >= 10000, display “Membership level: Platinum”.
- Step 4: Otherwise, if purchases >= 5000, display “Membership level: Gold”.
- Step 5: Otherwise, if purchases >= 1000, display “Membership level: Silver”.
- Step 6: Otherwise, display “Membership level: Regular”.
- Step 7: End.

Pseudocode:

START
INPUT purchases
IF purchases >= 10000 THEN
 DISPLAY "Platinum"
ELSE IF purchases >= 5000 THEN
 DISPLAY "Gold"
ELSE IF purchases >= 1000 THEN
 DISPLAY "Silver"
ELSE
 DISPLAY "Regular"
END IF
END

Flowchart:



Python Program:

```
purchases = float(input("Enter total purchases for the year: "))
if purchases >= 10000:
    print("Membership level: Platinum")
elif purchases >= 5000:
    print("Membership level: Gold")
elif purchases >= 1000:
    print("Membership level: Silver")
else:
    print("Membership level: Regular")
```

Sample Run 1:

Enter total purchases for the year: 12000
Membership level: Platinum

Sample Run 2:

Enter total purchases for the year: 6500
Membership level: Gold

Sample Run 3:

Enter total purchases for the year: 500
Membership level: Regular

Explanation: The structure guarantees that exactly one level is displayed. Because the conditions are checked from the highest amount downward, a customer with ₱12,000 is classified as Platinum even though ₱12,000 also satisfies the lower conditions.

Example 7.4: Positive, Negative, or Zero

Problem Statement: Ask the user for a number and classify it as positive, negative, or zero.

Input: a number.

Process: compare the number with zero using > and <.

Output: “Positive”, “Negative”, or “Zero”.

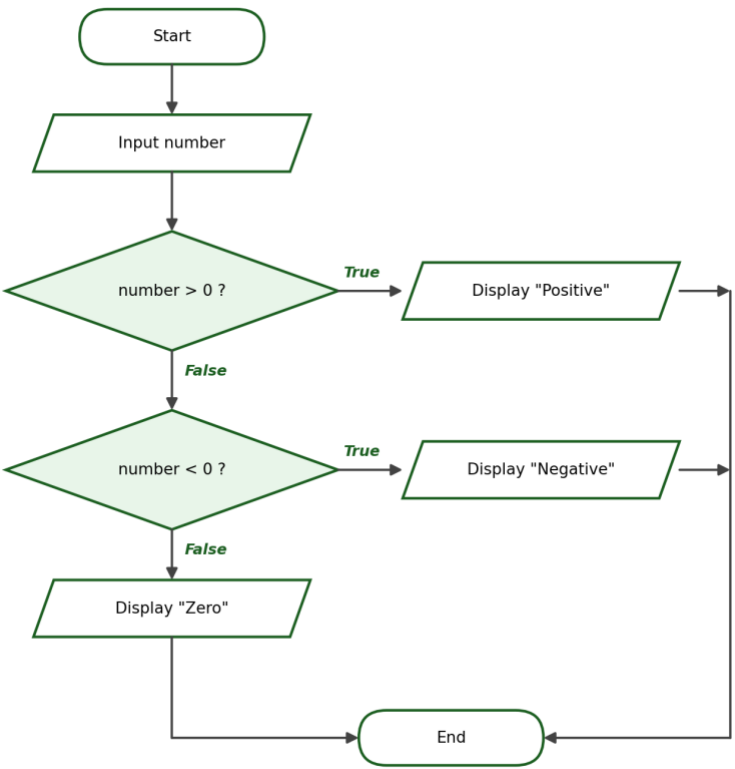
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter a number. Store it as number.
- Step 3: If number > 0, display “Positive”.
- Step 4: Otherwise, if number < 0, display “Negative”.
- Step 5: Otherwise, display “Zero”.
- Step 6: End.

Pseudocode:

```
START
INPUT number
IF number > 0 THEN
    DISPLAY "Positive"
ELSE IF number < 0 THEN
    DISPLAY "Negative"
ELSE
    DISPLAY "Zero"
END IF
END
```

Flowchart:



Python Program:

```
number = float(input("Enter a number: "))
if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

Sample Run 1:

```
Enter a number: 8
Positive
```

Sample Run 2:

```
Enter a number: -4
Negative
```

Sample Run 3:

```
Enter a number: 0
Zero
```

Explanation: A number that is neither greater than zero nor less than zero can only be zero, so the else block safely handles the last possibility without needing its own condition.

Checkpoint B: Predict the Output

Without running the code, write the exact output of each program. Answers are in the instructor’s answer key.

Program 1:

```
score = 70
if score >= 75:
    print("Passed")
else:
    print("Failed")
```

Program 2:

```
number = 10
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Program 3:

```
grade = 90
if grade >= 90:
    print("Excellent")
elif grade >= 75:
    print("Passed")
else:
    print("Failed")
```

Program 4:

```
x = 5
if x > 0:
    print("Positive")
print("Done")
```

Program 5:

```
age = 17
has_id = True
if age >= 18 and has_id:
    print("Allowed")
else:
    print("Not allowed")
```

Program 6:

```
a = 8
b = 8
if a > b:
    print("a is larger")
elif b > a:
    print("b is larger")
else:
    print("Equal")
```

Program 7:

```
price = 1200
if price >= 1000:
    discount = price * 0.10
    print(f"Discount: ₦{discount:.2f}")
else:
    print("No discount")
```

Program 8:

```
grade = 75
if grade > 90:
    print("Excellent")
elif grade >= 75:
    print("Passed")
else:
    print("Failed")
```

10. Nested if Statements

A nested if is a conditional statement placed inside another conditional statement. The inner condition is checked only when the outer condition is True. Nesting is useful when a second question only makes sense after the first question has been answered.

Example 10.1: Age and Identification Card

Problem Statement: A venue admits a person who is at least 18 years old and presents a valid ID. Ask for the age and whether the person has a valid ID, then display “Entry allowed”, “ID is required”, or “Entry not allowed”.

Input: the person’s age and an answer (yes/no) about the ID.

Process: check the age first; only when the age qualifies, check the ID.

Output: “Entry allowed”, “ID is required”, or “Entry not allowed”.

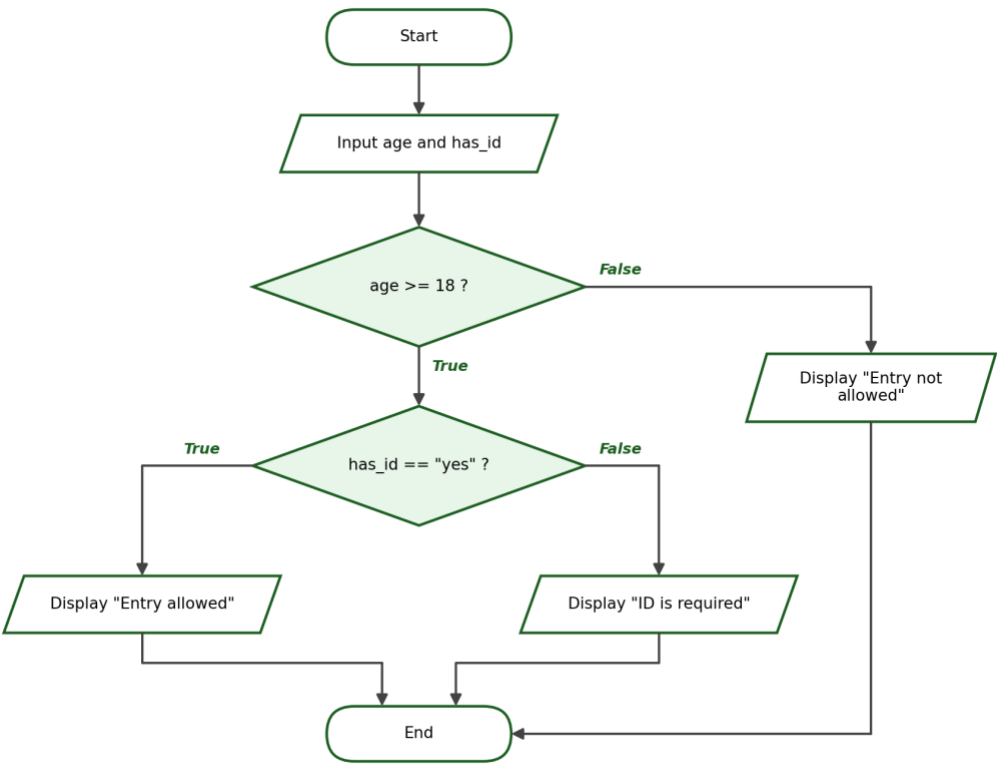
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the age. Store it as age.
- Step 3: Ask whether the person has a valid ID. Store the answer as has_id.
- Step 4: If age >= 18: if has_id equals “yes”, display “Entry allowed”; otherwise display “ID is required”.
- Step 5: If age < 18, display “Entry not allowed”.
- Step 6: End.

Pseudocode:

```
START
INPUT age
INPUT has_id
IF age >= 18 THEN
    IF has_id == "yes" THEN
        DISPLAY "Entry allowed"
    ELSE
        DISPLAY "ID is required"
    END IF
ELSE
    DISPLAY "Entry not allowed"
END IF
END
```

Flowchart:



Python Program:

```
age = int(input("Enter your age: "))
has_id = input("Do you have a valid ID (yes/no)? ")
if age >= 18:
    if has_id == "yes":
        print("Entry allowed")
    else:
        print("ID is required")
else:
    print("Entry not allowed")
```

Sample Run 1:

```
Enter your age: 20
Do you have a valid ID (yes/no)? yes
Entry allowed
```

Sample Run 2:

```
Enter your age: 20
Do you have a valid ID (yes/no)? no
ID is required
```

Sample Run 3:

```
Enter your age: 16
Do you have a valid ID (yes/no)? yes
Entry not allowed
```

Explanation: The inner if runs only when the outer condition age >= 18 is True. A 16-year-old is refused immediately regardless of the ID, which is exactly what the nesting expresses.

Example 10.2: Passing Grade and Completion of Requirements

Problem Statement: A student officially passes when the grade is at least 75 and all requirements are complete. Ask for the grade and whether the requirements are complete, then display “Passed and complete”, “Passed but incomplete”, or “Failed”.

Input: the grade and an answer (yes/no) about the requirements.

Process: check the grade first; only when the student passed, check the requirements.

Output: “Passed and complete”, “Passed but incomplete”, or “Failed”.

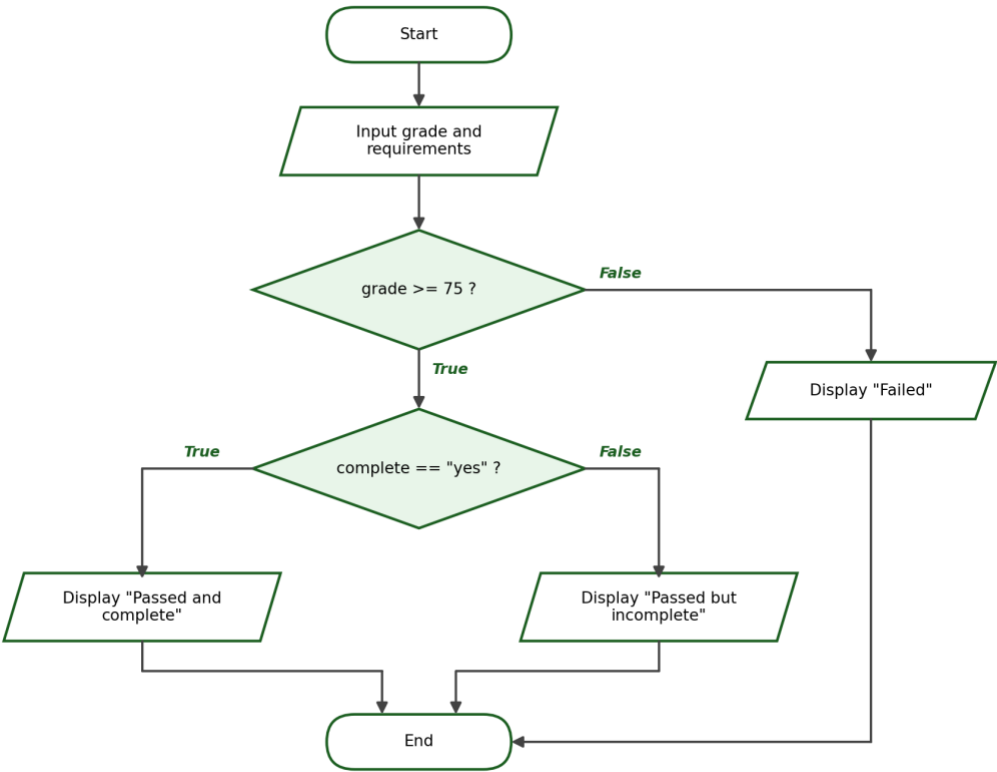
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the grade. Store it as grade.
- Step 3: Ask whether the requirements are complete. Store the answer as requirements_complete.
- Step 4: If grade >= 75: if requirements_complete equals “yes”, display “Passed and complete”; otherwise display “Passed but incomplete”.
- Step 5: If grade < 75, display “Failed”.
- Step 6: End.

Pseudocode:

```
START
INPUT grade
INPUT requirements_complete
IF grade >= 75 THEN
  IF requirements_complete == "yes" THEN
    DISPLAY "Passed and complete"
  ELSE
    DISPLAY "Passed but incomplete"
  END IF
ELSE
  DISPLAY "Failed"
END IF
END
```

Flowchart:



Python Program:

```
grade = float(input("Enter grade: "))
requirements_complete = input("Are the requirements complete (yes/no)? ")
if grade >= 75:
    if requirements_complete == "yes":
        print("Passed and complete")
    else:
        print("Passed but incomplete")
else:
    print("Failed")
```

Sample Run 1:

```
Enter grade: 85
Are the requirements complete (yes/no)? yes
Passed and complete
```

Sample Run 2:

```
Enter grade: 85
Are the requirements complete (yes/no)? no
Passed but incomplete
```

Sample Run 3:

```
Enter grade: 70
Are the requirements complete (yes/no)? yes
Failed
```

Explanation: Keep nested conditions simple and readable: one inner decision is usually enough at this level. Notice that a failing student is marked “Failed” even if the requirements are complete, because the inner question is only asked of students who passed.

11. Combining Multiple Conditions

Logical operators let one condition express what would otherwise need nesting. The nested structure of Example 10.1 can be shortened when the outputs for the two failing cases are the same:

```
age = int(input("Enter your age: "))
has_id = input("Do you have a valid ID (yes/no)? ")
if age >= 18 and has_id == "yes":
    print("Entry allowed")
else:
    print("Entry not allowed")
```

Use and when all requirements must be satisfied at the same time, use or when any single requirement is enough, and use not to reverse a condition. When combining several operators, use parentheses to make the intended grouping clear, for example: (age >= 18 and has_id == "yes") or accompanied == "yes".

12. Order of Conditions

In an if-elif-else structure, Python stops at the first condition that is True. If a broader condition is written before a narrower one, the narrower condition can never be reached. Study the following incorrect example:

```
grade = 95
if grade >= 75:
    print("Passed")
elif grade >= 90:
    print("Excellent")
```

Output:

```
Passed
```

“Excellent” will never be displayed, because every grade of at least 90 is also at least 75, so the first condition captures it first and the elif is never checked. The corrected version arranges the conditions from the narrowest (highest boundary) to the broadest:

```
grade = 95
if grade >= 90:
    print("Excellent")
elif grade >= 75:
    print("Passed")
else:
    print("Failed")
```

Output:

Excellent

The same mistake happens with any overlapping ranges. In the shipping example below, a 0.8 kg package is charged ₱180 instead of ₱50 because the broader condition weight <= 10 appears first:

```
weight = 0.8
if weight <= 10:
    print("Fee: 180")
elif weight <= 1:
    print("Fee: 50")
```

Output:

Fee: 180

Rule of thumb: when conditions use >=, arrange them from the highest boundary to the lowest; when conditions use <=, arrange them from the lowest boundary to the highest.

13. Common Conditional-Statement Errors

The table below lists the most common mistakes students make when writing conditional statements, together with the corrected forms.

Common Mistake	Incorrect Example	Correct Example
Using = instead of ==	if grade = 75:	if grade == 75:
Missing colon	if grade >= 75	if grade >= 75:
Incorrect indentation	if grade >= 75: print("Passed")	if grade >= 75: print("Passed")
Wrong order of conditions	if grade >= 75: ... elif grade >= 90: ...	if grade >= 90: ... elif grade >= 75: ...
Invalid variable name	2nd_grade = 80	second_grade = 80
Forgetting type conversion	grade = input("Grade: ") if grade >= 75:	grade = float(input("Grade: ")) if grade >= 75:
Comparing a string to a number	"75" == 75 (False)	int("75") == 75 (True)
Writing Else instead of else	Else:	else:
Writing ELIF instead of elif	ELIF grade >= 80:	elif grade >= 80:
Condition after else	else grade < 75:	else:

Python is case-sensitive: if, elif, and else must be written entirely in lowercase. Also remember that else never takes a condition — it simply catches every case that the earlier conditions did not.

Checkpoint C: Correct the Code

Each program below contains at least one error. Identify and correct the errors. Answers are in the instructor’s answer key.

Program 1:

```

grade = 75
if grade = 75:
    print("Exactly seventy-five")

```

Program 2:

```

number = 9
if number % 2 == 0
    print("Even")
else:
    print("Odd")

```

Program 3:

```

age = 20
if age >= 18:
    print("Adult")

```

Program 4:

```

score = 50
if score >= 75:
    print("Passed")
Else:
    print("Failed")

```

Program 5:

```

temp = 40
if temp >= 25:
    print("Warm")
elif temp >= 35:
    print("Hot")
else:
    print("Cold")

```

14. Flowcharts for Conditional Statements

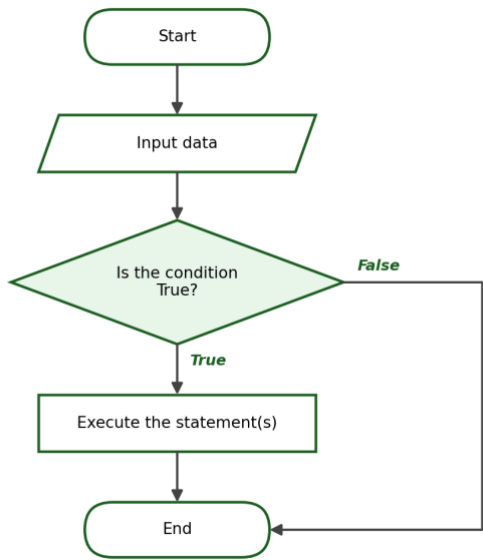
A flowchart is a diagram that shows the flow of a program using standard symbols. The symbols used in this module are listed below.

Symbol	Purpose
Oval	Start or End of the program
Parallelogram	Input or Output
Rectangle	Processing (computation or assignment)
Diamond	Decision (a condition with two or more paths)
Arrow	Direction of flow

A decision diamond must contain a question or a condition, and every path leaving the diamond must be labeled with the answer it represents, usually True and False (or Yes and No). For a basic if-else structure, the flow is: Start → Input → Decision → True/False branches → Output → End.

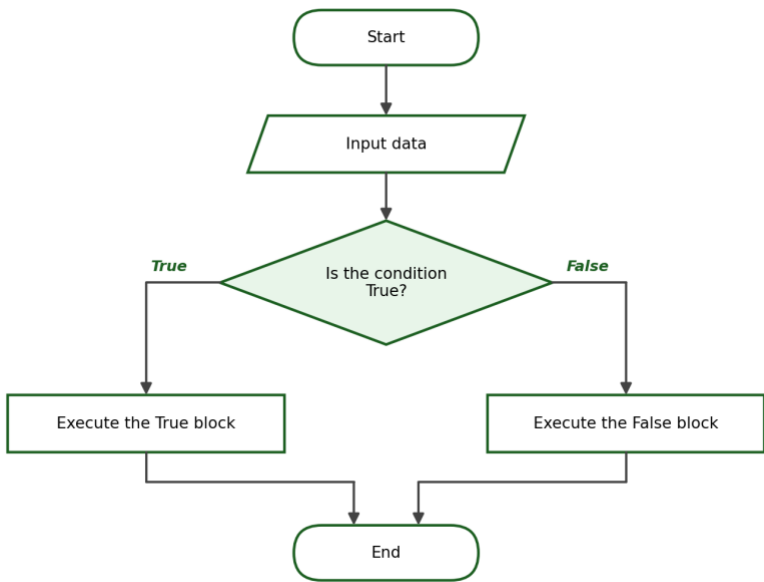
a. Flowchart of a Simple if

When the condition is False, the flow simply bypasses the statement block:



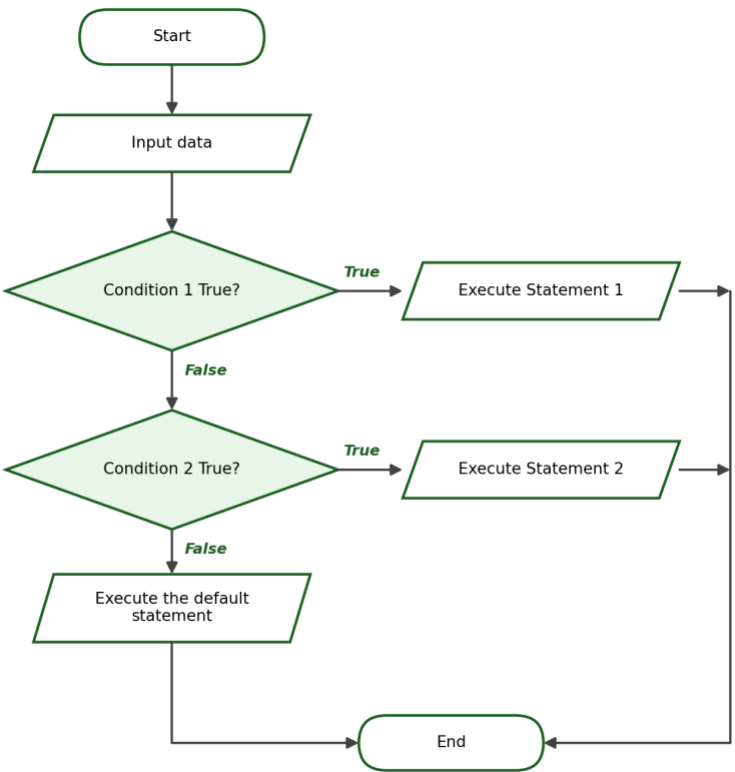
b. Flowchart of an if-else

Exactly one of the two branches executes before the flows merge again:



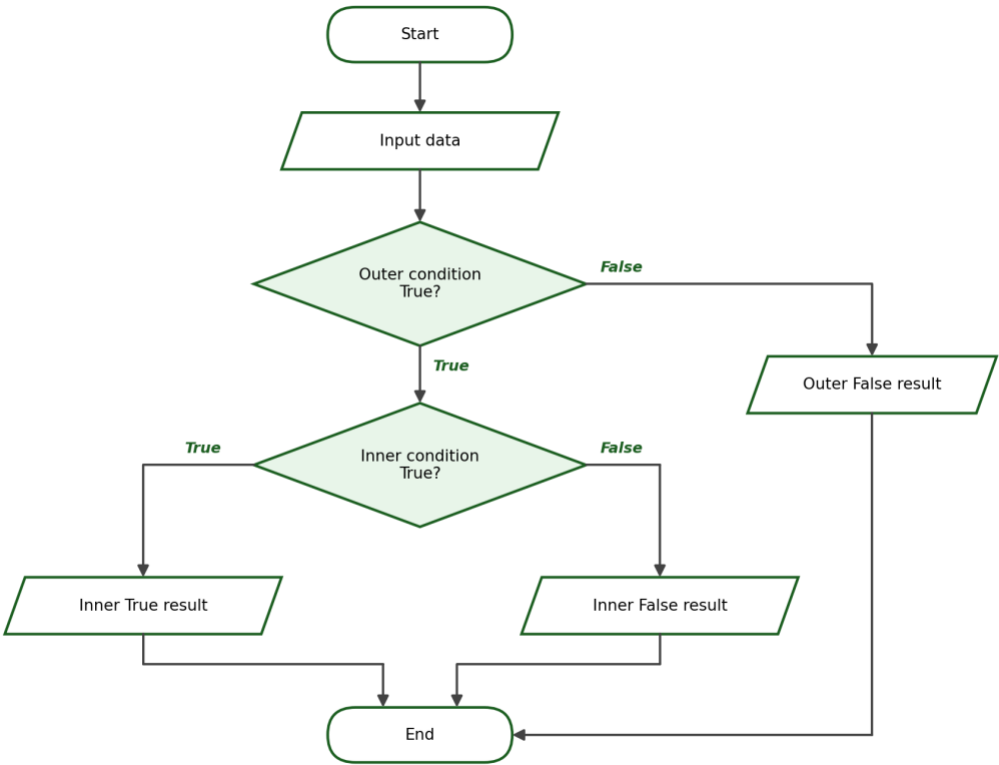
c. Flowchart of an if-elif-else

The conditions are tested one after another; the first True condition selects the path:



d. Flowchart of a Nested if

The inner decision is reached only through the True path of the outer decision:



15. Guided Problem-Solving Examples

Each guided example below follows the complete problem-solving process: problem analysis, IPO table, condition, algorithm, pseudocode, flowchart, Python program, sample runs with exact outputs, and a brief explanation. Follow the same process in the laboratory activities.

Guided Example 1: Pass or Fail

Problem Statement: Input a student’s grade. Display “Passed” if the grade is at least 75; otherwise, display “Failed”.

IPO Table:

Input	Process	Output
grade	Compare grade with 75 using >=	"Passed" or "Failed"

Problem Analysis: the program needs one input (the grade) and one decision. The boundary value is 75: a grade of exactly 75 must be classified as "Passed".

Condition: grade >= 75

Algorithm:

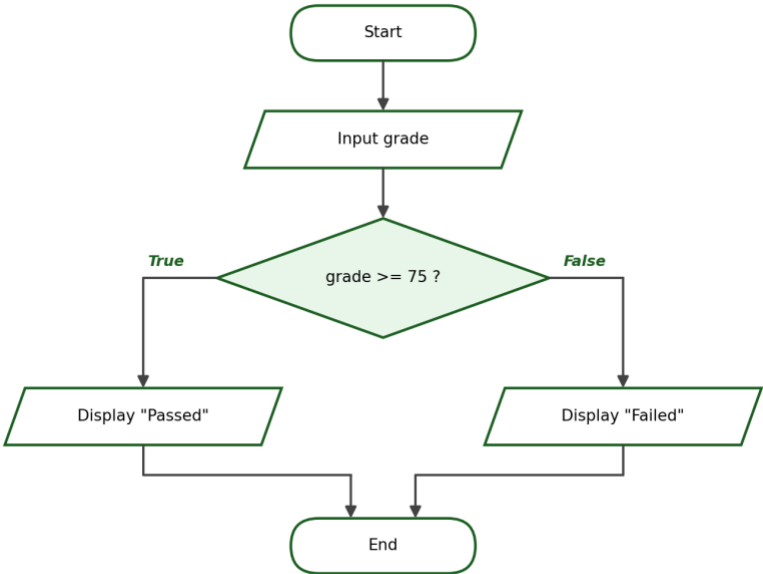
- Step 1: Start.
- Step 2: Ask the user to enter the grade. Store it as grade.
- Step 3: If grade >= 75, display "Passed"; otherwise, display "Failed".
- Step 4: End.

Pseudocode:

```

START
INPUT grade
IF grade >= 75 THEN
    DISPLAY "Passed"
ELSE
    DISPLAY "Failed"
END IF
END
  
```

Flowchart:



Python Program:

```

grade = float(input("Enter the student's grade: "))
if grade >= 75:
    print("Passed")
else:
    print("Failed")
  
```

Sample Run 1:

```

Enter the student's grade: 80
Passed
  
```

Sample Run 2:

Enter the student's grade: 74.9
Failed

Sample Run 3:

Enter the student's grade: 75
Passed

Explanation: The third sample run tests the boundary value. Since the operator is $\geq$, an input of exactly 75 satisfies the condition and the program correctly displays “Passed”.

Guided Example 2: Even or Odd

Problem Statement: Input an integer and determine whether it is even or odd.

IPO Table:

Input	Process	Output
number	Compute number % 2 and compare with 0	“Even” or “Odd”

Problem Analysis: a number is even when the remainder after dividing by 2 is zero. The modulo operator (%) gives that remainder.

Condition: number % 2 == 0

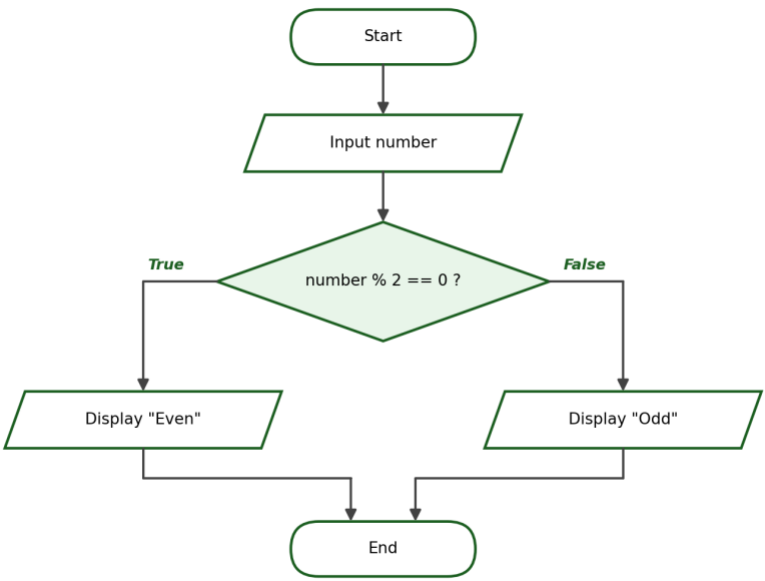
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter an integer. Store it as number.
- Step 3: If number % 2 equals 0, display “Even”; otherwise, display “Odd”.
- Step 4: End.

Pseudocode:

START
INPUT number
IF number MOD 2 == 0 THEN
 DISPLAY "Even"
ELSE
 DISPLAY "Odd"
END IF
END

Flowchart:



Python Program:

```
number = int(input("Enter an integer: "))
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Sample Run 1:

Enter an integer: 24
Even

Sample Run 2:

Enter an integer: 13
Odd

Sample Run 3:

Enter an integer: 0
Even

Explanation: Zero divided by 2 leaves no remainder, so 0 is classified as even — a boundary case worth testing. The input uses int() because evenness applies to whole numbers.

Guided Example 3: Positive, Negative, or Zero

Problem Statement: Input a number and classify it as positive, negative, or zero.

IPO Table:

Input	Process	Output
number	Compare number with 0 using > and <	“Positive”, “Negative”, or “Zero”

Problem Analysis: three outcomes require an if-elif-else structure. Two comparisons are enough: whatever is neither positive nor negative must be zero.

Condition: number > 0, then number < 0, otherwise zero

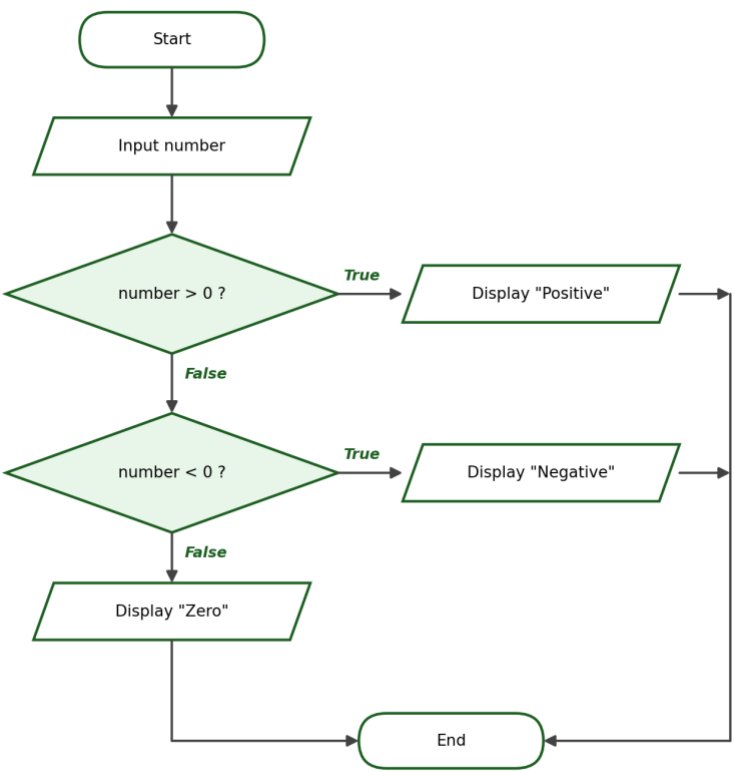
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter a number. Store it as number.
- Step 3: If number > 0, display “Positive”.
- Step 4: Otherwise, if number < 0, display “Negative”.
- Step 5: Otherwise, display “Zero”.
- Step 6: End.

Pseudocode:

```
START
INPUT number
IF number > 0 THEN
    DISPLAY "Positive"
ELSE IF number < 0 THEN
    DISPLAY "Negative"
ELSE
    DISPLAY "Zero"
END IF
END
```

Flowchart:



Python Program:

```
number = float(input("Enter a number: "))
if number > 0:
    print("Positive")
elif number < 0:
    print("Negative")
else:
    print("Zero")
```

Sample Run 1:

Enter a number: 15
Positive

Sample Run 2:

Enter a number: -7.5
Negative

Sample Run 3:

Enter a number: 0
Zero

Explanation: The three test cases cover all three paths of the structure. The else block needs no condition because zero is the only remaining possibility.

Guided Example 4: Largest of Two Numbers

Problem Statement: Input two numbers and display the larger value. If the values are equal, display “The two numbers are equal”.

IPO Table:

Input	Process	Output
num1, num2	Compare num1 and num2	The larger number, or an equality message

Problem Analysis: there are three possible outcomes: the first number is larger, the second number is larger, or they are equal. An if-elif-else structure handles all three.

Condition: num1 > num2, then num2 > num1, otherwise equal

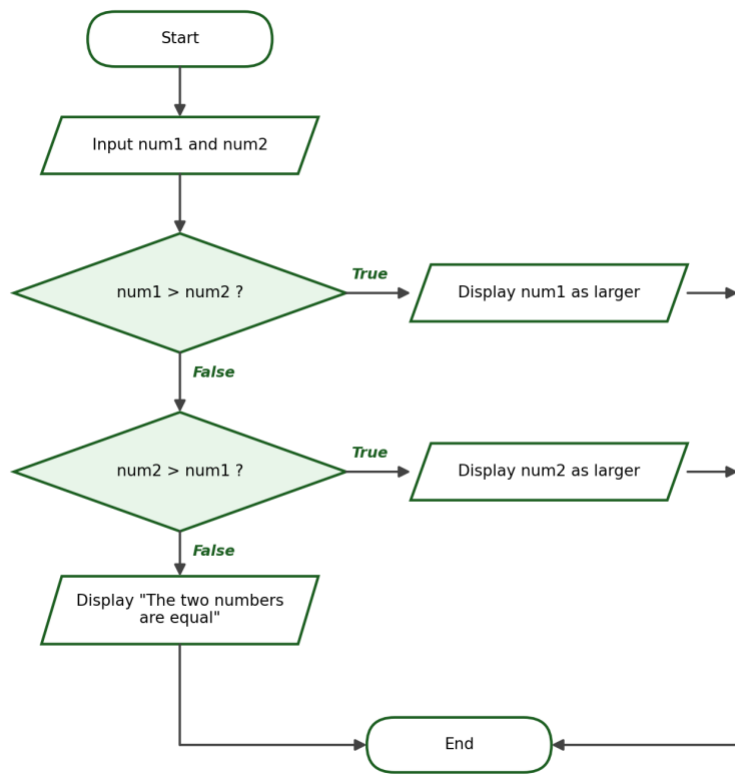
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the first number. Store it as num1.
- Step 3: Ask the user to enter the second number. Store it as num2.
- Step 4: If num1 > num2, display num1 as the larger number.
- Step 5: Otherwise, if num2 > num1, display num2 as the larger number.
- Step 6: Otherwise, display “The two numbers are equal”.
- Step 7: End.

Pseudocode:

START
INPUT num1
INPUT num2
IF num1 > num2 THEN
 DISPLAY "The larger number is " + num1
ELSE IF num2 > num1 THEN
 DISPLAY "The larger number is " + num2
ELSE
 DISPLAY "The two numbers are equal"
END IF
END

Flowchart:



Python Program:

```
num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
if num1 > num2:
    print(f"The larger number is {num1}")
elif num2 > num1:
    print(f"The larger number is {num2}")
else:
    print("The two numbers are equal")
```

Sample Run 1:

```
Enter the first number: 8
Enter the second number: 12
The larger number is 12.0
```

Sample Run 2:

```
Enter the first number: 20
Enter the second number: 5
The larger number is 20.0
```

Sample Run 3:

```
Enter the first number: 9
Enter the second number: 9
The two numbers are equal
```

Explanation: Because the inputs are converted with float(), Python displays them with a decimal point (12.0). Handling the equal case explicitly prevents the program from wrongly declaring either value “larger”.

Guided Example 5: Discount Eligibility

Problem Statement: A customer receives a 10% discount if the total purchase is at least ₱1,000. Compute and display the discount and the final amount.

IPO Table:

Input	Process	Output
purchase	If purchase >= 1000: discount = purchase × 0.10; final = purchase – discount	Discount and final amount

Problem Analysis: the decision determines whether a discount applies; the True branch needs two computations (discount and final amount) before the output.

Condition: purchase >= 1000

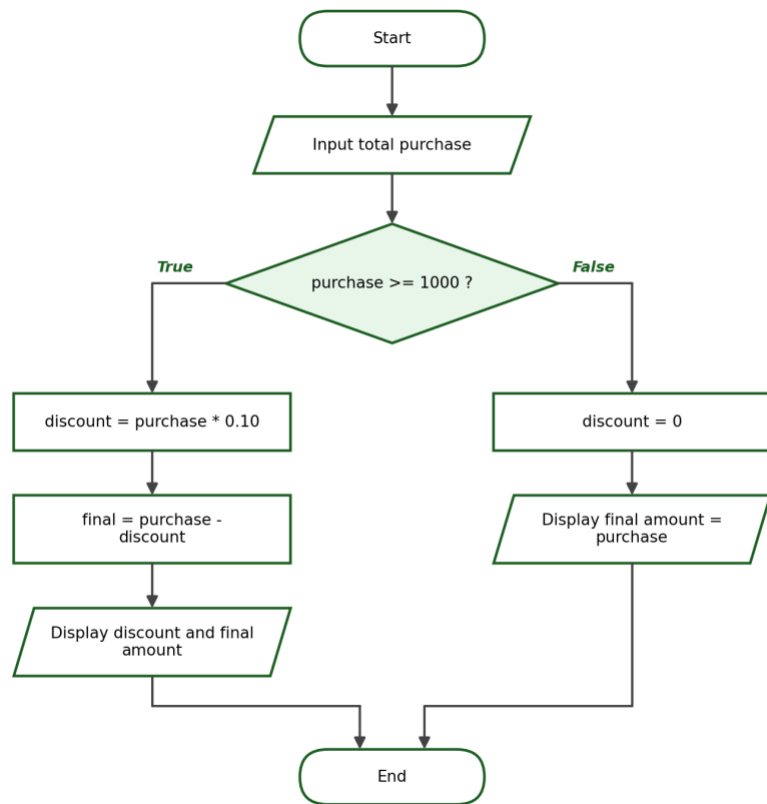
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the total purchase. Store it as purchase.
- Step 3: If purchase >= 1000, compute discount = purchase × 0.10 and final_amount = purchase – discount, then display both.
- Step 4: Otherwise, display “No discount.” and a final amount equal to the purchase.
- Step 5: End.

Pseudocode:

```
START
INPUT purchase
IF purchase >= 1000 THEN
    discount = purchase * 0.10
    final_amount = purchase - discount
    DISPLAY discount, final_amount
ELSE
    DISPLAY "No discount."
    DISPLAY purchase
END IF
END
```

Flowchart:



Python Program:

```
purchase = float(input("Enter total purchase: "))
if purchase >= 1000:
    discount = purchase * 0.10
    final_amount = purchase - discount
    print(f"Discount: ₱{discount:.2f}")
    print(f"Final Amount: ₱{final_amount:.2f}")
else:
    print("No discount.")
    print(f"Final Amount: ₱{purchase:.2f}")
```

Sample Run 1:

```
Enter total purchase: 2500
Discount: ₱250.00
Final Amount: ₱2250.00
```

Sample Run 2:

```
Enter total purchase: 999.99
No discount.
Final Amount: ₱999.99
```

Sample Run 3:

```
Enter total purchase: 1000
Discount: ₱100.00
Final Amount: ₱900.00
```

Explanation: Check the arithmetic: 10% of ₱2,500 is ₱250, leaving ₱2,250. The boundary input ₱1,000 earns the discount because the operator is >=. Currency is displayed with two decimal places using the format specifier :.2f.

Guided Example 6: Grade Classification

Problem Statement: Input a grade and display the appropriate classification: 90–100 Excellent, 85–89 Very Good, 80–84 Good, 75–79 Passed, below 75 Failed.

IPO Table:

Input	Process	Output
grade	Compare grade against 90, 85, 80, and 75, from highest to lowest	One of the five classifications

Problem Analysis: five outcomes require four conditions plus an else. Arranging the boundaries from highest to lowest keeps every condition simple.

Condition: grade >= 90, then grade >= 85, then grade >= 80, then grade >= 75, otherwise Failed

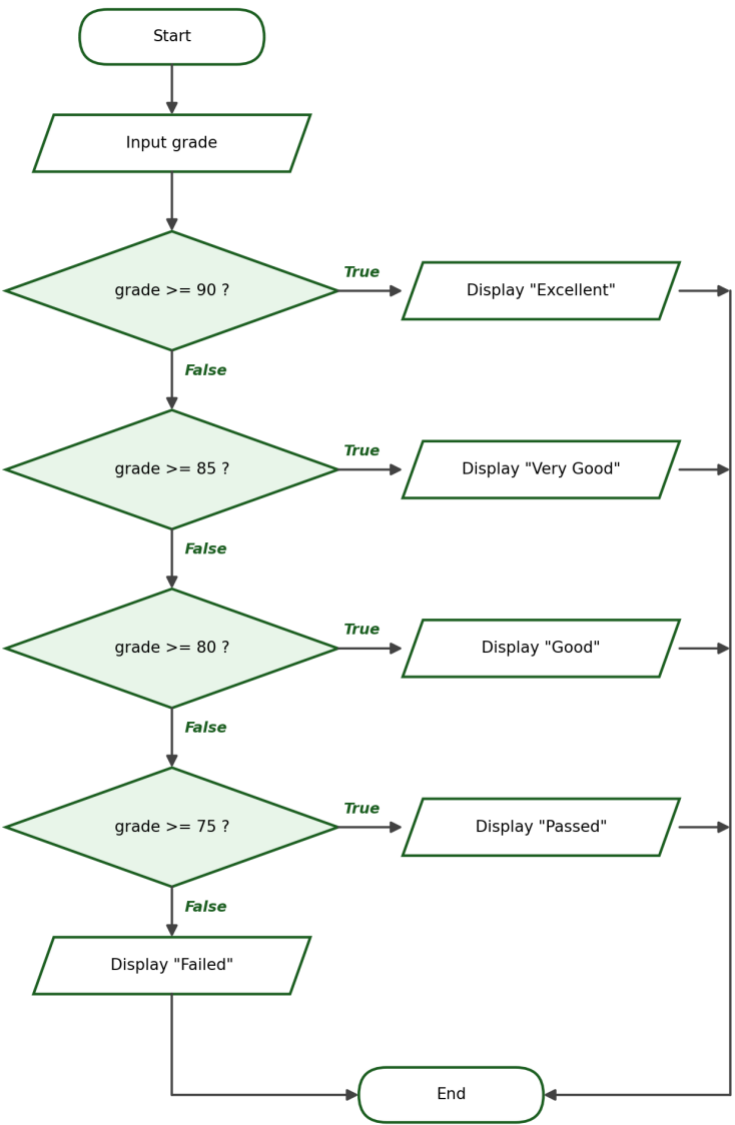
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the grade. Store it as grade.
- Step 3: If grade >= 90, display “Excellent”.
- Step 4: Otherwise, if grade >= 85, display “Very Good”.
- Step 5: Otherwise, if grade >= 80, display “Good”.
- Step 6: Otherwise, if grade >= 75, display “Passed”.
- Step 7: Otherwise, display “Failed”.
- Step 8: End.

Pseudocode:

```
START
INPUT grade
IF grade >= 90 THEN
    DISPLAY "Excellent"
ELSE IF grade >= 85 THEN
    DISPLAY "Very Good"
ELSE IF grade >= 80 THEN
    DISPLAY "Good"
ELSE IF grade >= 75 THEN
    DISPLAY "Passed"
ELSE
    DISPLAY "Failed"
END IF
END
```

Flowchart:



Python Program:

```
grade = float(input("Enter the grade: "))
if grade >= 90:
    print("Excellent")
elif grade >= 85:
    print("Very Good")
elif grade >= 80:
    print("Good")
elif grade >= 75:
    print("Passed")
else:
    print("Failed")
```

Sample Run 1:

```
Enter the grade: 93
Excellent
```

Sample Run 2:

```
Enter the grade: 85
Very Good
```

Sample Run 3:

Enter the grade: 79
Passed

Sample Run 4:

Enter the grade: 74
Failed

Explanation: The sample runs test an ordinary value (93), a boundary value (85, the lowest “Very Good”), the top of the “Passed” range (79), and a failing value (74). Testing boundaries confirms that each range starts and ends exactly where the table says it should.

PART B. WEEK 5 LABORATORY MANUAL

1. Laboratory Title

Developing Python Programs Using Conditional Statements

2. Laboratory Objectives

At the end of the laboratory session, students should be able to:

1. Write and test Python programs that use if, if-else, if-elif-else, and nested conditional statements to solve given problems.
2. Prepare the complete problem-solving documents for each program: IPO table, algorithm, pseudocode, and flowchart.
3. Commit and push the completed Week 5 source-code files to the GitHub repository using the terminal or Visual Studio Code.

3. Required Tools

- Desktop or laptop computer
- Python 3
- Visual Studio Code
- Terminal
- Git
- GitHub account

4. Laboratory Rules

1. Perform the activity using Visual Studio Code and the terminal.
2. Run and test every Python program before submitting it.
3. Use meaningful variable names.
4. Follow proper Python indentation (four spaces per level).
5. Do not edit files directly through the GitHub website.
6. Save all source-code files in the required Week 5 repository folder.
7. Submit original work. Copying another student's code is an academic-integrity violation.

5. Laboratory Preparation: Review Activity

Before starting the activities, create a file named review.py, type the program below, and run it. This short exercise reviews variables, numeric input, type conversion, comparison operators, logical operators, and formatted output from Week 4.

```
quiz1 = float(input("Enter Quiz 1 score: "))
quiz2 = float(input("Enter Quiz 2 score: "))
average = (quiz1 + quiz2) / 2
print(f"Average: {average:.2f}")
print("At least 75?", average >= 75)
print("Both above 70?", quiz1 > 70 and quiz2 > 70)
```

Predict the three output lines for the inputs 80 and 74 before running the program, then check your prediction. If your prediction was wrong, review Week 4 before continuing.

6. Laboratory Activity 1 — Basic Conditional Statements

Solve the following problems using a simple if or an if-else statement. For every problem, prepare an IPO table, an algorithm, a pseudocode, a flowchart, the Python source code, and a screenshot or copy of the program output.

1. Positive Number. Input a number. Display “Positive” if the number is greater than zero.
2. Passed or Failed. Input a grade. Display “Passed” if the grade is at least 75; otherwise, display “Failed”.
3. Even or Odd. Input an integer. Display “Even” if the number is even; otherwise, display “Odd”.
4. Overtime Check. Input the number of hours an employee worked this week. If the hours exceed 40, display the number of overtime hours rendered; otherwise, display “No overtime rendered”.

Save the programs as activity1/problem1.py to activity1/problem4.py.

7. Laboratory Activity 2 — Multiple Conditions (if-elif-else)

Solve the following problems using an if-elif-else statement. The condition ranges are complete and do not overlap; follow them exactly. Prepare the same requirements as in Activity 1.

1. Grade Classification. Input a grade and classify it: 90–100 Excellent, 85–89 Very Good, 80–84 Good, 75–79 Passed, below 75 Failed.
2. Temperature Classification. Input the temperature in Celsius and classify it: 35 and above Hot, 25 to below 35 Warm, 15 to below 25 Cool, below 15 Cold.
3. Age-Group Classification. Input an age and classify it: 0–12 Child, 13–19 Teenager, 20–59 Adult, 60 and above Senior Citizen.
4. Shipping-Fee Classification. Input a package weight in kilograms and display the fee: up to 1 kg ₱50, above 1 kg up to 5 kg ₱100, above 5 kg up to 10 kg ₱180, above 10 kg ₱250.

Save the programs as activity2/problem1.py to activity2/problem4.py.

8. Laboratory Activity 3 — Logical Operators

Solve the following problems using and, or, or not. Prepare the same requirements as in Activity 1.

1. Scholarship Eligibility. A student qualifies for a scholarship when the general average is at least 90 and the attendance is at least 95%. Input both values and display “Qualified for scholarship” or “Not qualified”.
2. Entrance Eligibility. A person may enter an event when the age is at least 18 or a guardian approval is submitted. Input the age and the approval answer (yes/no) and display “Entry allowed” or “Entry not allowed”.
3. Account-Access Check. Access is granted when the account status is active and the account does not have a violation. Input both answers (yes/no) and display “Access granted” or “Access denied”.

Save the programs as activity3/problem1.py to activity3/problem3.py.

9. Laboratory Activity 4 — Nested Conditional Statements

Solve the following problems using a nested if with one inner decision only. Prepare the same requirements as in Activity 1.

1. Grade and Requirements. Input a grade and whether the requirements are complete (yes/no). Display “Passed and complete” when the grade is at least 75 and the requirements are complete, “Passed but incomplete” when the grade is at least 75 but the requirements are incomplete, and “Failed” otherwise.
2. Membership and Purchase. Input whether the customer is a member (yes/no) and the purchase amount. Members receive a 10% discount when the purchase is at least ₱2,000 and a 5% discount otherwise; non-members receive no discount. Display the discount (if any) and the final amount.

Save the programs as activity4/problem1.py and activity4/problem2.py.

10. Laboratory Activity 5 — Complete Problem-Solving Tasks

The instructor will assign at least one of the following problems. Each assigned problem requires the complete set: problem analysis, IPO table, algorithm, pseudocode, flowchart, Python program, three test cases, and the expected outputs.

1. Electricity-Bill Classification. Input the monthly consumption in kWh and classify the usage: up to 100 kWh Low usage, above 100 up to 300 kWh Moderate usage, above 300 kWh High usage.
2. Parking Fee. A parking lot charges a flat fee of ₱50 for the first 3 hours and ₱20 for every additional hour. Input the number of hours parked (whole hours) and display the total fee.
3. Delivery Charge. Input the delivery distance in kilometers and display the charge: up to 3 km ₱40, above 3 km up to 10 km ₱70, above 10 km ₱120.
4. Ticket Price. Input the age of the customer and display the price: 4 years old and below free admission, 5–12 ₱150, 13–59 ₱300, 60 and above ₱240 (senior discounted price).
5. Largest of Three Numbers. Input three numbers and display the largest value.
6. Employee Overtime Pay. Input the hours worked this week and the hourly rate. If the hours exceed 40, the pay is $40 \times \text{rate}$ plus $1.25 \times \text{rate}$ for every hour beyond 40, classified as “With overtime”; otherwise the pay is $\text{hours} \times \text{rate}$, classified as “Regular”. Display the classification and the total pay.

Save the assigned programs inside the activity5 folder using the problem number, for example activity5/problem2.py.

11. Required Test Cases

For every complete problem-solving activity, prepare at least three test cases: one where the first condition is satisfied, one where another condition is satisfied, and one boundary-value test case. Record them in a table like the following example for the passing grade problem:

Test Case	Input	Expected Result
1	74	Failed
2	75	Passed
3	90	Passed

Boundary testing is important because most conditional-logic errors happen exactly at the edge of a range — for example, writing `>` instead of `>=` silently misclassifies the boundary value while every other input still looks correct. A program is only trustworthy after its boundaries have been tested.

12. Git and GitHub Submission Workflow

Organize your Week 5 repository folder exactly as shown below, replacing “surname” with your own surname:

```
cpp102-week5-surname/  
├── activity1/  
│   ├── problem1.py  
│   ├── problem2.py  
│   ├── problem3.py  
│   └── problem4.py  
├── activity2/  
├── activity3/  
├── activity4/  
├── activity5/  
└── README.md
```

After completing and testing your programs, open the terminal inside the repository folder and run the basic workflow:

```
git status
git add .
git commit -m "Complete CPP102 Week 5 conditional statement activities"
git push
```

Troubleshooting: if Git reports an index-lock error (a message mentioning `.git/index.lock`), close other Git operations and remove the leftover lock file with:

```
rm .git/index.lock
```

Important: use the terminal or Visual Studio Code for the Git workflow. Do not upload or edit the Python files directly through the GitHub website.

README Requirement

The README.md file at the root of the repository must contain:

- Student name
- Section
- Week number
- List of completed activities
- Short reflection (three to five sentences)

13. Reflection Questions

Answer the following questions in the reflection section of your README.md:

1. What is the difference between an if statement and an if-else statement?
2. Why is the order of conditions important in an if-elif-else structure?
3. How did flowcharting help you develop and check your Python program?

PART C. CLOSING

1. Summary

Conditional statements let a program choose its path of execution based on conditions that evaluate to True or False. Conditions are built from comparison operators (`==`, `!=`, `>`, `<`, `>=`, `<=`) and can be combined with the logical operators `and`, `or`, and `not`. Python identifies the statements belonging to a conditional block through indentation, using four spaces per level. The simple if executes a block only when the condition is True; the if-else always selects exactly one of two blocks; the if-elif-else selects one result among several conditions, checked from top to bottom; and a nested if places one decision inside another. Conditions with overlapping ranges must be ordered correctly, and every decision structure should be tested at its boundary values. In a flowchart, a decision appears as a diamond whose outgoing paths are labeled True and False.

2. References

Textbooks and Library References

- Gaddis, T. Starting Out with Programming Logic and Design (latest available edition). Pearson.
- Farrell, J. Programming Logic and Design (latest available edition). Cengage Learning.
- Deitel, P., & Deitel, H. Intro to Python for Computer Science and Data Science (latest available edition). Pearson.

Online References

- Python Software Foundation. The Python Tutorial: More Control Flow Tools. <https://docs.python.org/3/tutorial/controlflow.html>

- Python Software Foundation. The Python Language Reference: Compound Statements. https://docs.python.org/3/reference/compound_stmts.html
- Git Documentation. <https://git-scm.com/doc>
- GitHub Docs. <https://docs.github.com>

PART D. INSTRUCTOR’S COMPLETE ANSWER KEY

For instructor use only. This section contains the complete answers to the lecture checkpoint activities, the guided examples, and all laboratory activities. Every program below was executed and its outputs verified in Python 3.

1. Checkpoint A Answer Key — Evaluate the Condition

No.	Expression	Answer	Reason
1	10 > 5	True	10 is greater than 5
2	8 == 8	True	Both values are equal
3	12 != 12	False	The values are equal, so “not equal” is false
4	75 >= 75	True	75 is equal to 75
5	20 < 10 or 5 == 5	True	The right side (5 == 5) is true; or needs only one true side
6	not (7 > 3)	False	7 > 3 is True; not reverses it
7	15 <= 14	False	15 is greater than 14
8	3 > 1 and 2 > 5	False	The right side (2 > 5) is false; and needs both sides true
9	not (4 == 4) and 10 > 2	False	not True is False, so the and is False
10	6 != 7 or 8 < 3	True	The left side (6 != 7) is true

2. Checkpoint B Answer Key — Predict the Output

Program 1 — exact output:

```
Failed
```

Why: score is 70; 70 >= 75 is False, so the else block runs.

Program 2 — exact output:

```
Even
```

Why: 10 % 2 equals 0, so the condition is True.

Program 3 — exact output:

```
Excellent
```

Why: 90 >= 90 is True; the first condition already matches.

Program 4 — exact output:

```
Positive
Done
```

Why: The first line prints because 5 > 0; “Done” is not indented, so it always prints.

Program 5 — exact output:

```
Not allowed
```

Why: age >= 18 is False; and requires both sides to be true.

Program 6 — exact output:

```
Equal
```

Why: Neither a > b nor b > a is true, so the else block runs.

Program 7 — exact output:

Discount: ₱120.00

Why: 1200 >= 1000 is True; 10% of 1200 is 120, printed with two decimals.

Program 8 — exact output:

Passed

Why: 75 > 90 is False, but 75 >= 75 is True — a boundary-value trap.

3. Checkpoint C Answer Key — Correct the Code

Program 1

Error: The condition uses the assignment operator = instead of the comparison operator ==.

Corrected program:

```
grade = 75
if grade == 75:
    print("Exactly seventy-five")
```

Output of the corrected program:

Exactly seventy-five

Program 2

Error: The if line is missing a colon at the end.

Corrected program:

```
number = 9
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Output of the corrected program:

Odd

Program 3

Error: The statement under the if is not indented.

Corrected program:

```
age = 20
if age >= 18:
    print("Adult")
```

Output of the corrected program:

Adult

Program 4

Error: Else is capitalized; Python keywords are lowercase.

Corrected program:

```
score = 50
if score >= 75:
    print("Passed")
else:
    print("Failed")
```

Output of the corrected program:

Failed

Program 5

Error: The conditions are in the wrong order: temp >= 25 captures 40 before temp >= 35 is ever checked, so “Hot” can never be displayed.

Corrected program:

```
temp = 40
if temp >= 35:
    print("Hot")
elif temp >= 25:
    print("Warm")
else:
    print("Cold")
```

Output of the corrected program:

Hot

4. Guided Examples — Verification Notes

The six guided examples in Part A, Section 15 are presented with their complete solutions (IPO table, condition, algorithm, pseudocode, flowchart, program, and sample runs). All sample outputs were verified by execution. The verified boundary results are summarized below for quick reference during discussion:

Guided Example	Boundary Input	Verified Output
1. Pass or Fail	75	Passed
2. Even or Odd	0	Even
3. Positive, Negative, or Zero	0	Zero
4. Largest of Two Numbers	9 and 9	The two numbers are equal
5. Discount Eligibility	1000	Discount: ₱100.00 / Final Amount: ₱900.00
6. Grade Classification	85	Very Good

5. Laboratory Activity 1 Answer Key — Basic Conditional Statements

Activity 1, Problem 1: Positive Number

IPO Table:

Input	Process	Output
number	Compare number with 0 using >	“Positive” (when the condition is True)

Condition / Formula:
 $\text{number} > 0$

Algorithm:

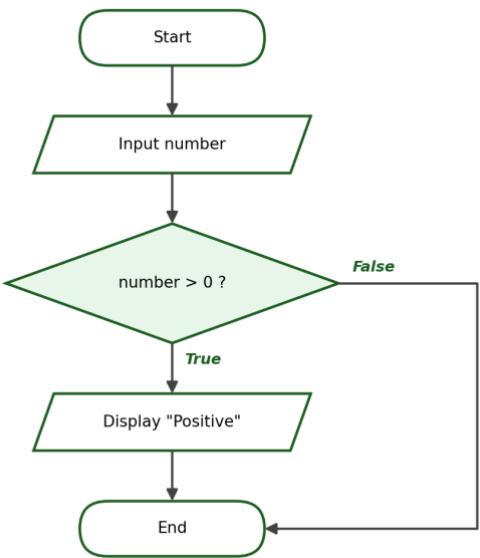
- Step 1: Start.
- Step 2: Ask the user to enter a number. Store it as number.
- Step 3: If $\text{number} > 0$, display “Positive”.
- Step 4: End.

Pseudocode:

```

START
INPUT number
IF number > 0 THEN
    DISPLAY "Positive"
END IF
END
    
```

Flowchart:



Python Program:

```

number = float(input("Enter a number: "))
if number > 0:
    print("Positive")
    
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	5	Positive
2	-5	(no output — the condition is False)
3	0.1	Positive

Explanation: A simple if is enough because the problem asks for output only in one case. The boundary of the condition is 0 itself: an input of 0 produces no output because $0 > 0$ is False.

Activity 1, Problem 2: Passed or Failed

IPO Table:

Input	Process	Output
grade	Compare grade with 75 using >=	"Passed" or "Failed"

Condition / Formula:
grade >= 75

Algorithm:

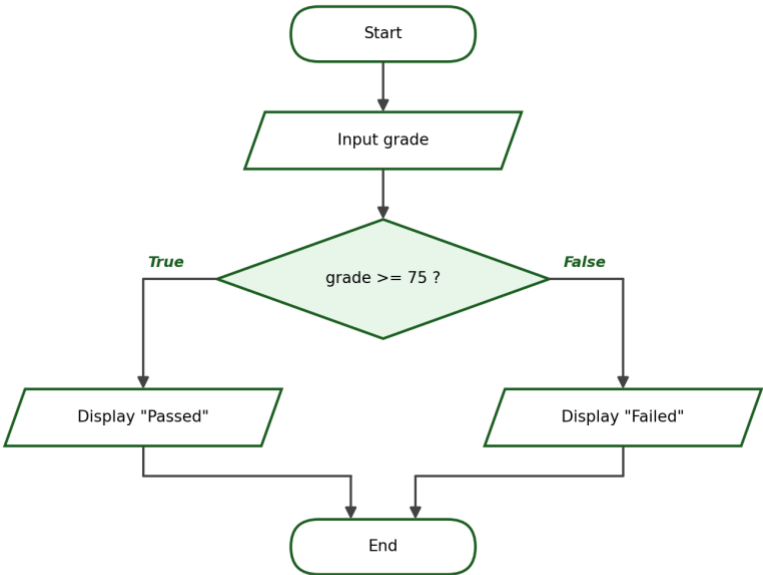
- Step 1: Start.
- Step 2: Ask the user to enter the grade. Store it as grade.
- Step 3: If grade >= 75, display "Passed"; otherwise, display "Failed".
- Step 4: End.

Pseudocode:

```

START
INPUT grade
IF grade >= 75 THEN
    DISPLAY "Passed"
ELSE
    DISPLAY "Failed"
END IF
END
  
```

Flowchart:



Python Program:

```

grade = float(input("Enter your grade: "))
if grade >= 75:
    print("Passed")
else:
    print("Failed")
  
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	74	Failed
2	75	Passed
3	90	Passed

Explanation: The boundary case 75 must display “Passed” because the operator is >=. A student who writes > instead of >= fails exactly this test case.

Activity 1, Problem 3: Even or Odd

IPO Table:

Input	Process	Output
number	Compute number % 2 and compare with 0	“Even” or “Odd”

Condition / Formula: number % 2 == 0

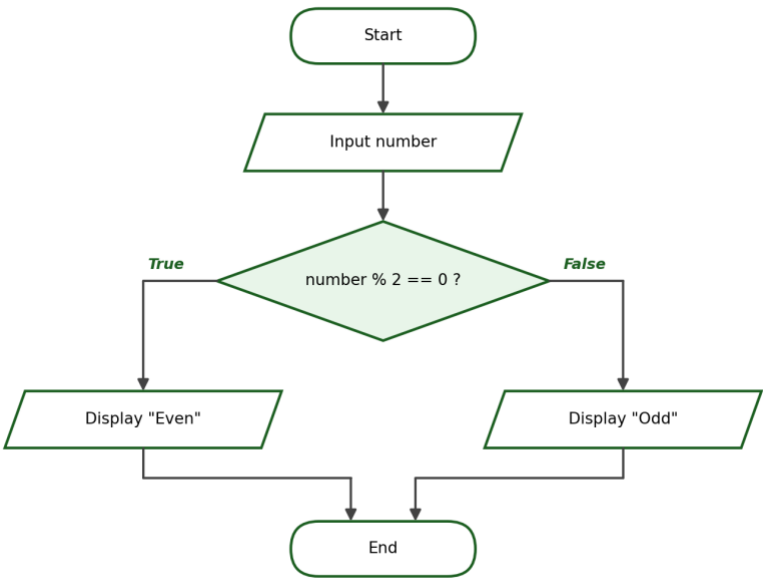
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter an integer. Store it as number.
- Step 3: If number % 2 equals 0, display “Even”; otherwise, display “Odd”.
- Step 4: End.

Pseudocode:

```
START
INPUT number
IF number MOD 2 == 0 THEN
    DISPLAY "Even"
ELSE
    DISPLAY "Odd"
END IF
END
```

Flowchart:



Python Program:

```
number = int(input("Enter an integer: "))
if number % 2 == 0:
    print("Even")
else:
    print("Odd")
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	4	Even
2	9	Odd
3	0	Even

Explanation: int() is used because evenness is defined for whole numbers. Zero is even — a boundary case that confirms the condition uses the remainder, not the size, of the number.

Activity 1, Problem 4: Overtime Check

IPO Table:

Input	Process	Output
hours	If hours > 40, compute overtime = hours – 40	Overtime hours, or “No overtime rendered”

Condition / Formula: hours > 40; overtime = hours – 40

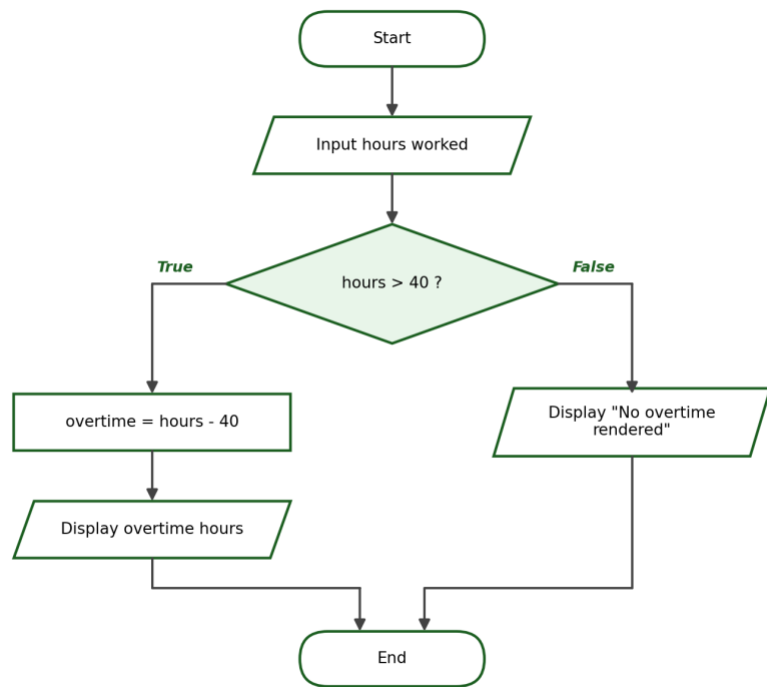
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the hours worked. Store it as hours.
- Step 3: If hours > 40, compute overtime = hours – 40 and display it.
- Step 4: Otherwise, display “No overtime rendered”.
- Step 5: End.

Pseudocode:

```
START
INPUT hours
IF hours > 40 THEN
    overtime = hours - 40
    DISPLAY "Overtime rendered: " + overtime
ELSE
    DISPLAY "No overtime rendered"
END IF
END
```

Flowchart:



Python Program:

```
hours = float(input("Enter hours worked this week: "))
if hours > 40:
    overtime = hours - 40
    print(f'Overtime rendered: {overtime:.1f} hours")
else:
    print("No overtime rendered")
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	45	Overtime rendered: 5.0 hours
2	38	No overtime rendered
3	40	No overtime rendered

Explanation: The condition uses > (strictly greater) because exactly 40 hours is a full regular week with no overtime — the boundary test case 40 confirms this.

6. Laboratory Activity 2 Answer Key — Multiple Conditions

Activity 2, Problem 1: Grade Classification

IPO Table:

Input	Process	Output
grade	Compare grade against 90, 85, 80, 75 from highest to lowest	Excellent / Very Good / Good / Passed / Failed

Condition / Formula: grade >= 90, grade >= 85, grade >= 80, grade >= 75, else Failed

Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the grade. Store it as grade.
- Step 3: If grade >= 90, display “Excellent”.
- Step 4: Otherwise, if grade >= 85, display “Very Good”.
- Step 5: Otherwise, if grade >= 80, display “Good”.

Step 6: Otherwise, if grade >= 75, display “Passed”.

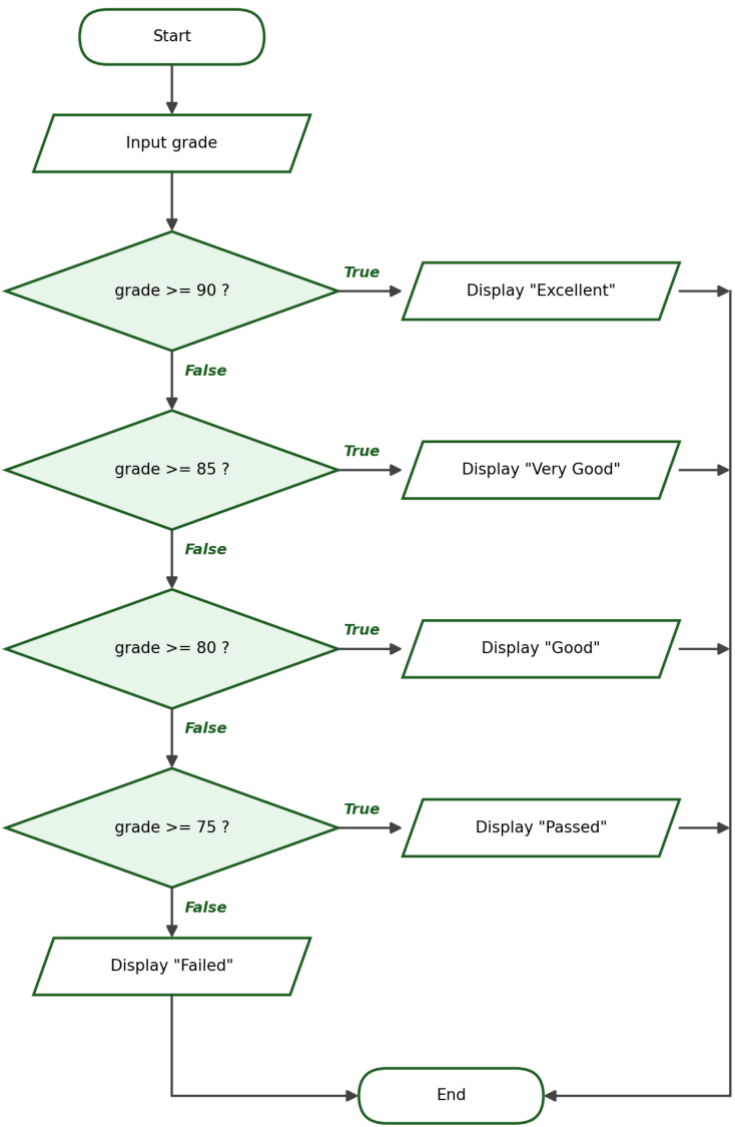
Step 7: Otherwise, display “Failed”.

Step 8: End.

Pseudocode:

```
START
INPUT grade
IF grade >= 90 THEN
    DISPLAY "Excellent"
ELSE IF grade >= 85 THEN
    DISPLAY "Very Good"
ELSE IF grade >= 80 THEN
    DISPLAY "Good"
ELSE IF grade >= 75 THEN
    DISPLAY "Passed"
ELSE
    DISPLAY "Failed"
END IF
END
```

Flowchart:



Python Program:

```
grade = float(input("Enter the grade: "))
if grade >= 90:
    print("Excellent")
elif grade >= 85:
    print("Very Good")
elif grade >= 80:
    print("Good")
elif grade >= 75:
    print("Passed")
else:
    print("Failed")
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	92	Excellent
2	83	Good
3	75	Passed (boundary)

Explanation: The boundaries are checked from highest to lowest so that each elif is reached only by grades below all previous boundaries. Accept solutions with equivalent range checks as long as the five classifications are produced correctly.

Activity 2, Problem 2: Temperature Classification

IPO Table:

Input	Process	Output
temperature	Compare temperature against 35, 25, 15 from highest to lowest	Hot / Warm / Cool / Cold

Condition / Formula: temperature >= 35, temperature >= 25, temperature >= 15, else Cold

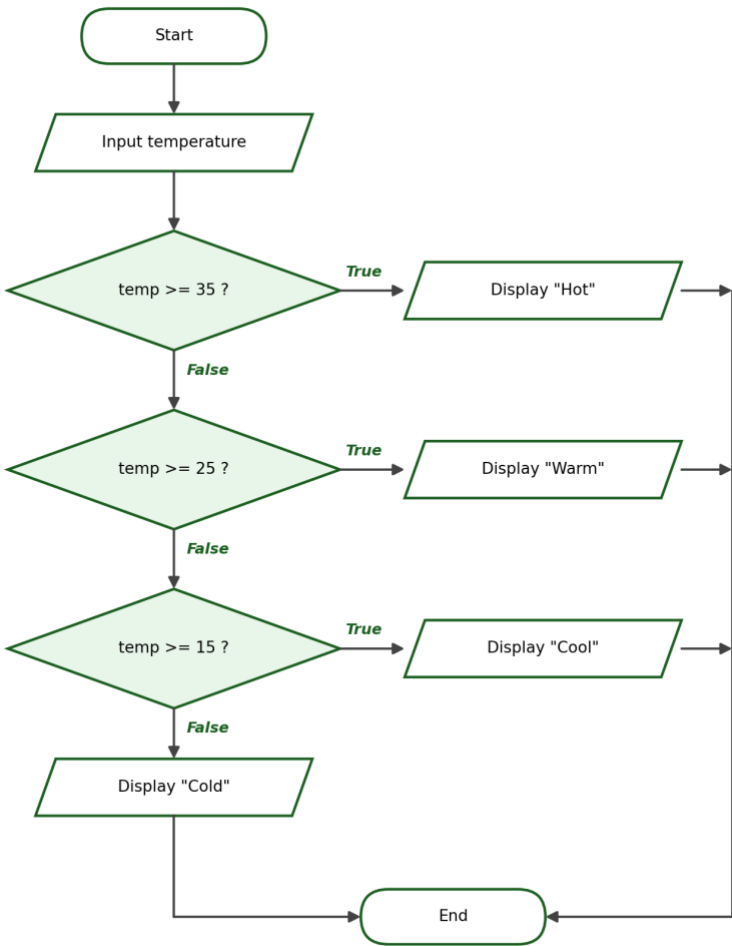
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the temperature. Store it as temperature.
- Step 3: If temperature >= 35, display “Hot”.
- Step 4: Otherwise, if temperature >= 25, display “Warm”.
- Step 5: Otherwise, if temperature >= 15, display “Cool”.
- Step 6: Otherwise, display “Cold”.
- Step 7: End.

Pseudocode:

```
START
INPUT temperature
IF temperature >= 35 THEN
    DISPLAY "Hot"
ELSE IF temperature >= 25 THEN
    DISPLAY "Warm"
ELSE IF temperature >= 15 THEN
    DISPLAY "Cool"
ELSE
    DISPLAY "Cold"
END IF
END
```

Flowchart:



Python Program:

```
temperature = float(input("Enter the temperature in Celsius: "))
if temperature >= 35:
    print("Hot")
elif temperature >= 25:
    print("Warm")
elif temperature >= 15:
    print("Cool")
else:
    print("Cold")
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	38	Hot
2	20	Cool
3	35	Hot (boundary)

Explanation: The boundary value 35 belongs to “Hot” because the range is “35 and above”. Each lower range needs no upper-bound check — failing the earlier conditions already guarantees it.

Activity 2, Problem 3: Age-Group Classification

IPO Table:

Input	Process	Output
age	Compare age against 12, 19, 59 from lowest to highest using <=	Child / Teenager / Adult / Senior Citizen

Condition / Formula: age <= 12, age <= 19, age <= 59, else Senior Citizen

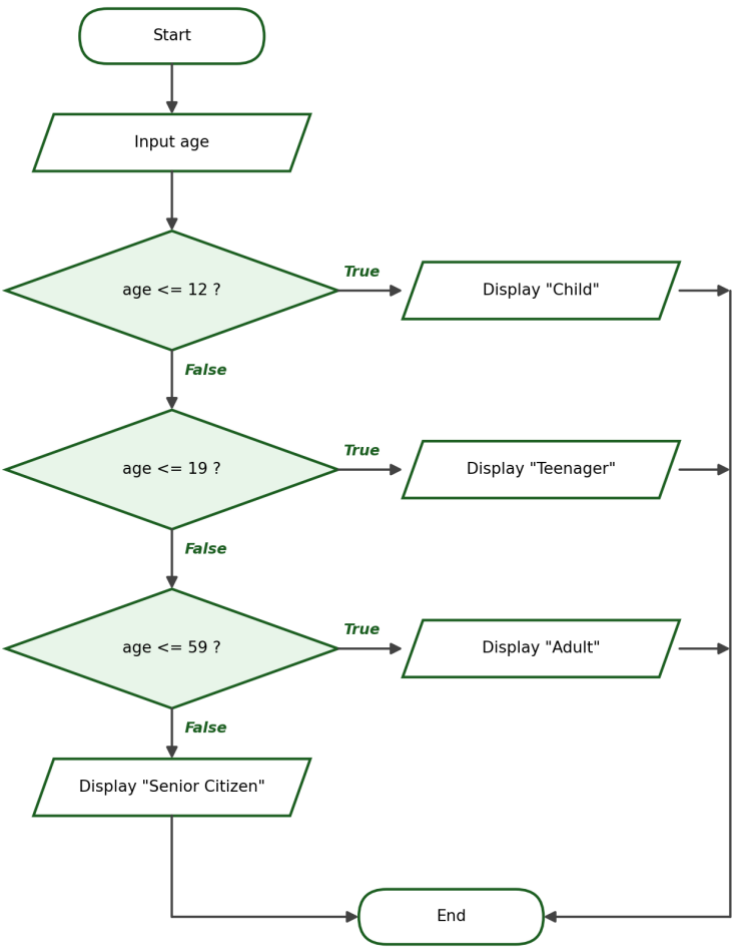
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the age. Store it as age.
- Step 3: If age <= 12, display “Child”.
- Step 4: Otherwise, if age <= 19, display “Teenager”.
- Step 5: Otherwise, if age <= 59, display “Adult”.
- Step 6: Otherwise, display “Senior Citizen”.
- Step 7: End.

Pseudocode:

```
START
INPUT age
IF age <= 12 THEN
    DISPLAY "Child"
ELSE IF age <= 19 THEN
    DISPLAY "Teenager"
ELSE IF age <= 59 THEN
    DISPLAY "Adult"
ELSE
    DISPLAY "Senior Citizen"
END IF
END
```

Flowchart:



Python Program:

```

age = int(input("Enter the age: "))
if age <= 12:
    print("Child")
elif age <= 19:
    print("Teenager")
elif age <= 59:
    print("Adult")
else:
    print("Senior Citizen")

```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	10	Child
2	35	Adult
3	60	Senior Citizen (boundary)

Explanation: Because these conditions use <=, they are arranged from the lowest boundary to the highest — the mirror image of the >= rule used in the grade problem.

Activity 2, Problem 4: Shipping-Fee Classification

IPO Table:

Input	Process	Output
weight	Compare weight against 1, 5, 10 from lowest to highest; select the fee	Shipping fee (₱50 / ₱100 / ₱180 / ₱250)

Condition / Formula: weight <= 1 → 50; weight <= 5 → 100; weight <= 10 → 180; else 250

Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the weight in kilograms. Store it as weight.
- Step 3: If weight <= 1, set fee = 50.
- Step 4: Otherwise, if weight <= 5, set fee = 100.
- Step 5: Otherwise, if weight <= 10, set fee = 180.
- Step 6: Otherwise, set fee = 250.
- Step 7: Display the shipping fee.
- Step 8: End.

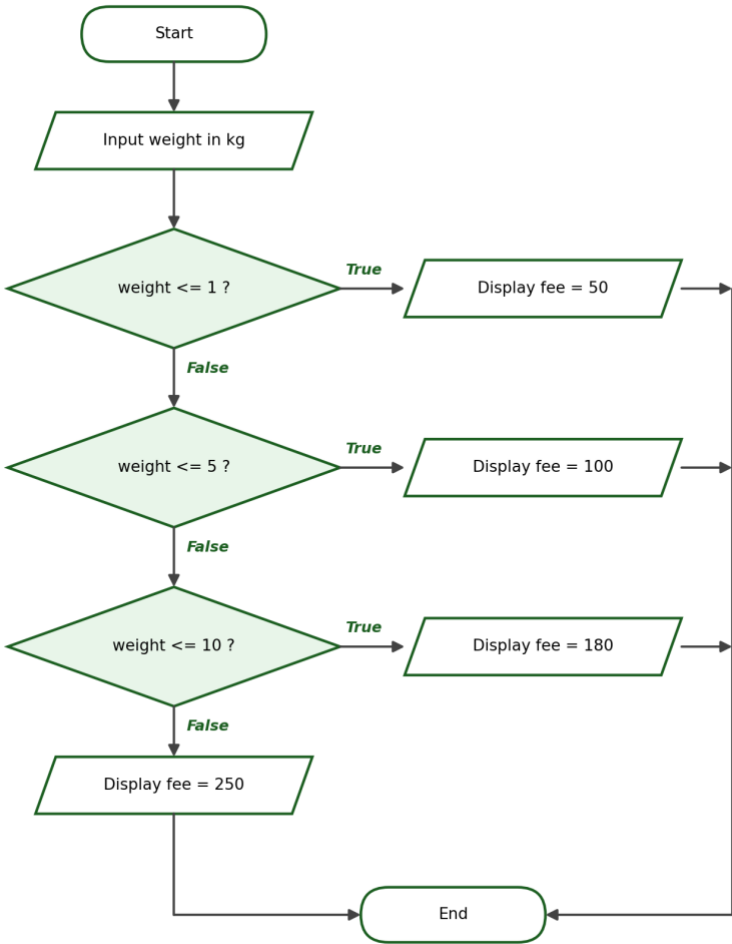
Pseudocode:

```

START
INPUT weight
IF weight <= 1 THEN
    fee = 50
ELSE IF weight <= 5 THEN
    fee = 100
ELSE IF weight <= 10 THEN
    fee = 180
ELSE
    fee = 250
END IF
DISPLAY "Shipping fee: " + fee
END

```

Flowchart:



Python Program:

```
weight = float(input("Enter package weight in kg: "))
if weight <= 1:
    fee = 50
elif weight <= 5:
    fee = 100
elif weight <= 10:
    fee = 180
else:
    fee = 250
print(f"Shipping fee: ₱{fee:.2f}")
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	0.5	Shipping fee: ₱50.00
2	7	Shipping fee: ₱180.00
3	5	Shipping fee: ₱100.00 (boundary)

Explanation: This solution shows that a decision structure can assign a value instead of printing immediately; a single formatted print at the end then handles the output for every branch.

7. Laboratory Activity 3 Answer Key — Logical Operators

Activity 3, Problem 1: Scholarship Eligibility

IPO Table:

Input	Process	Output
grade, attendance	Check grade >= 90 and attendance >= 95	"Qualified for scholarship" or "Not qualified"

Condition / Formula: grade >= 90 and attendance >= 95

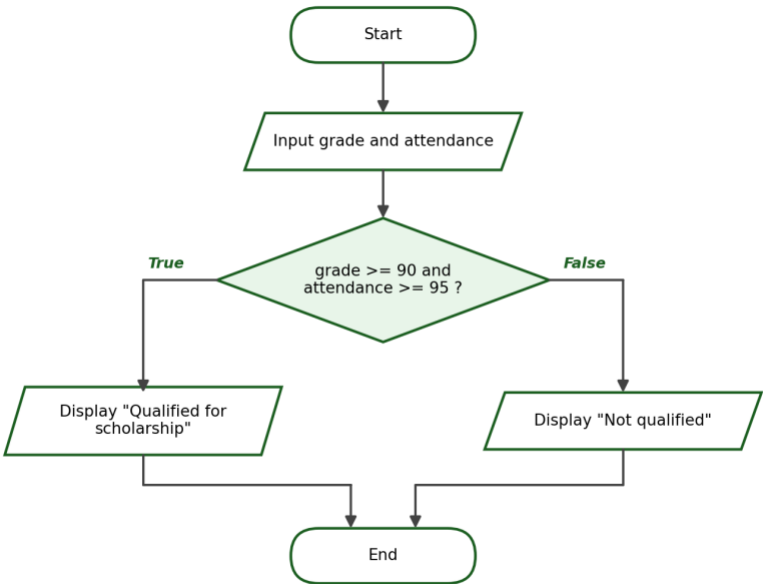
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the general average. Store it as grade.
- Step 3: Ask the user to enter the attendance percentage. Store it as attendance.
- Step 4: If grade >= 90 and attendance >= 95, display "Qualified for scholarship".
- Step 5: Otherwise, display "Not qualified".
- Step 6: End.

Pseudocode:

```
START
INPUT grade
INPUT attendance
IF grade >= 90 AND attendance >= 95 THEN
    DISPLAY "Qualified for scholarship"
ELSE
    DISPLAY "Not qualified"
END IF
END
```

Flowchart:



Python Program:

```
grade = float(input("Enter general average: "))
attendance = float(input("Enter attendance percentage: "))
if grade >= 90 and attendance >= 95:
    print("Qualified for scholarship")
else:
    print("Not qualified")
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	92 and 97	Qualified for scholarship
2	95 and 90	Not qualified
3	90 and 95	Qualified for scholarship (boundary)

Explanation: and requires both requirements at the same time: a high average with low attendance (second test) is not enough. The boundary test uses both minimum values at once.

Activity 3, Problem 2: Entrance Eligibility

IPO Table:

Input	Process	Output
age, approval	Check age >= 18 or approval == "yes"	"Entry allowed" or "Entry not allowed"

Condition / Formula: age >= 18 or approval == "yes"

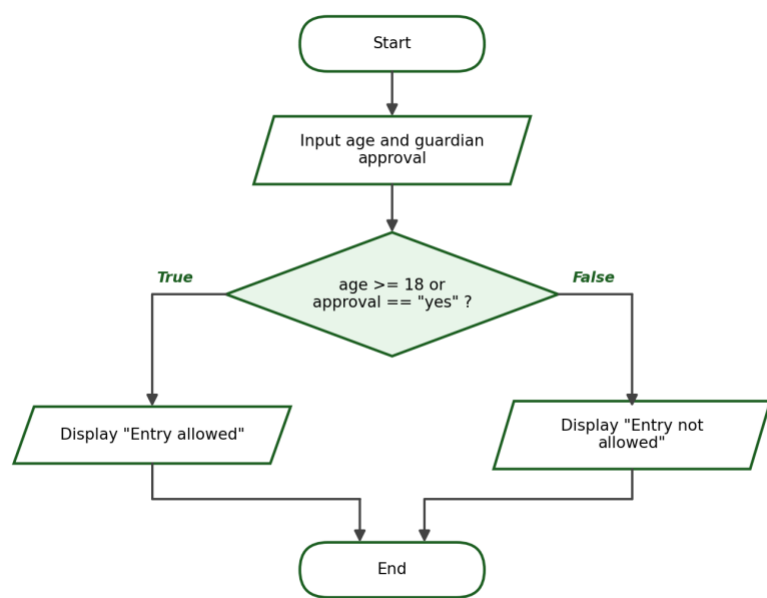
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the age. Store it as age.
- Step 3: Ask whether a guardian approval is submitted. Store the answer as approval.
- Step 4: If age >= 18 or approval equals "yes", display "Entry allowed".
- Step 5: Otherwise, display "Entry not allowed".
- Step 6: End.

Pseudocode:

```
START
INPUT age
INPUT approval
IF age >= 18 OR approval == "yes" THEN
    DISPLAY "Entry allowed"
ELSE
    DISPLAY "Entry not allowed"
END IF
END
```

Flowchart:



Python Program:

```

age = int(input("Enter your age: "))
approval = input("Is a guardian approval submitted (yes/no)? ")
if age >= 18 or approval == "yes":
    print("Entry allowed")
else:
    print("Entry not allowed")

```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	19 and no	Entry allowed
2	15 and yes	Entry allowed
3	17 and no	Entry not allowed (boundary)

Explanation: or grants entry when either requirement is met: an adult without approval and a minor with approval are both admitted. Only when both sides are false is entry refused.

Activity 3, Problem 3: Account-Access Check

IPO Table:

Input	Process	Output
is_active, has_violation	Check is_active == "yes" and not (has_violation == "yes")	"Access granted" or "Access denied"

Condition / Formula: is_active == "yes" and not (has_violation == "yes")

Algorithm:

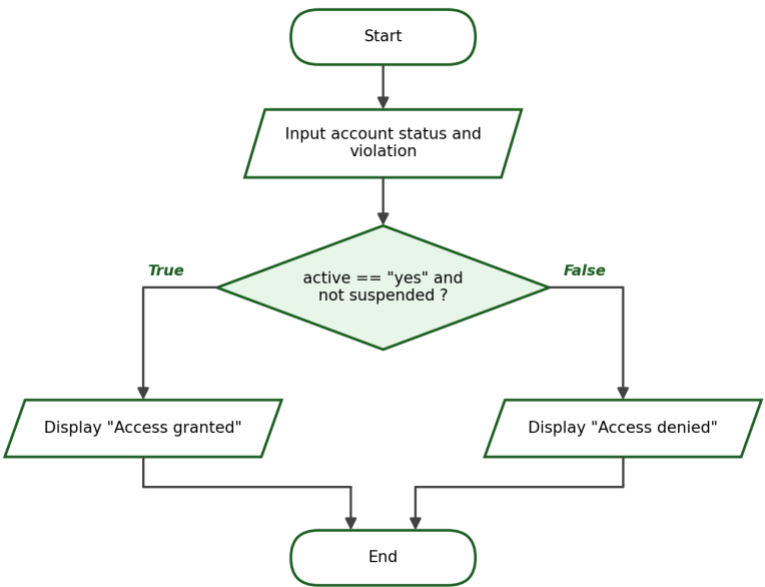
- Step 1: Start.
- Step 2: Ask whether the account is active. Store the answer as is_active.
- Step 3: Ask whether the account has a violation. Store the answer as has_violation.
- Step 4: If is_active equals "yes" and has_violation does not equal "yes", display "Access granted".
- Step 5: Otherwise, display "Access denied".
- Step 6: End.

Pseudocode:

```

START
INPUT is_active
INPUT has_violation
IF is_active == "yes" AND NOT (has_violation == "yes") THEN
    DISPLAY "Access granted"
ELSE
    DISPLAY "Access denied"
END IF
END
    
```

Flowchart:



Python Program:

```

is_active = input("Is the account active (yes/no)? ")
has_violation = input("Does the account have a violation (yes/no)? ")
if is_active == "yes" and not (has_violation == "yes"):
    print("Access granted")
else:
    print("Access denied")
    
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	yes and no	Access granted
2	yes and yes	Access denied
3	no and no	Access denied

Explanation: not reverses the violation check: access requires the account to be active and to have no violation. Accept the equivalent form `has_violation != "yes"` as a correct alternative.

8. Laboratory Activity 4 Answer Key — Nested Conditional Statements

Activity 4, Problem 1: Grade and Requirements

IPO Table:

Input	Process	Output
grade, requirements_complete	Check grade >= 75; inside the True branch, check the requirements answer	“Passed and complete”, “Passed but incomplete”, or “Failed”

Condition / Formula: outer: grade >= 75; inner: requirements_complete == "yes"

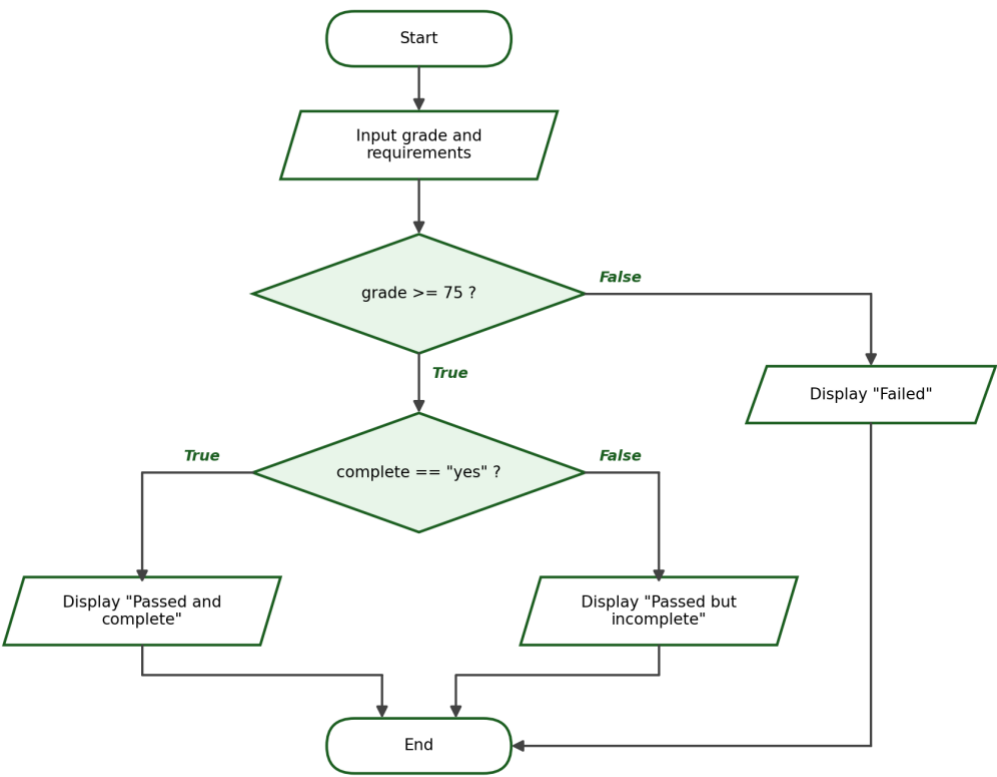
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the grade. Store it as grade.
- Step 3: Ask whether the requirements are complete. Store the answer as requirements_complete.
- Step 4: If grade >= 75: if requirements_complete equals "yes", display "Passed and complete"; otherwise display "Passed but incomplete".
- Step 5: If grade < 75, display "Failed".
- Step 6: End.

Pseudocode:

```
START
INPUT grade
INPUT requirements_complete
IF grade >= 75 THEN
    IF requirements_complete == "yes" THEN
        DISPLAY "Passed and complete"
    ELSE
        DISPLAY "Passed but incomplete"
    END IF
ELSE
    DISPLAY "Failed"
END IF
END
```

Flowchart:



Python Program:

```

grade = float(input("Enter grade: "))
requirements_complete = input("Are the requirements complete (yes/no)? ")
if grade >= 75:
    if requirements_complete == "yes":
        print("Passed and complete")
    else:
        print("Passed but incomplete")
else:
    print("Failed")

```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	88 and yes	Passed and complete
2	88 and no	Passed but incomplete
3	74 and yes	Failed (boundary)

Explanation: The requirements question matters only for passing students, which is exactly what the nesting expresses. The last test confirms that a failing grade is “Failed” regardless of the requirements.

Activity 4, Problem 2: Membership and Purchase

IPO Table:

Input	Process	Output
member, purchase	If member: discount is 10% when purchase >= 2000, otherwise 5%; compute final amount	Discount and final amount, or the purchase amount for non-members

Condition / Formula: outer: member == "yes"; inner: purchase >= 2000; discount = purchase × 0.10 or purchase × 0.05

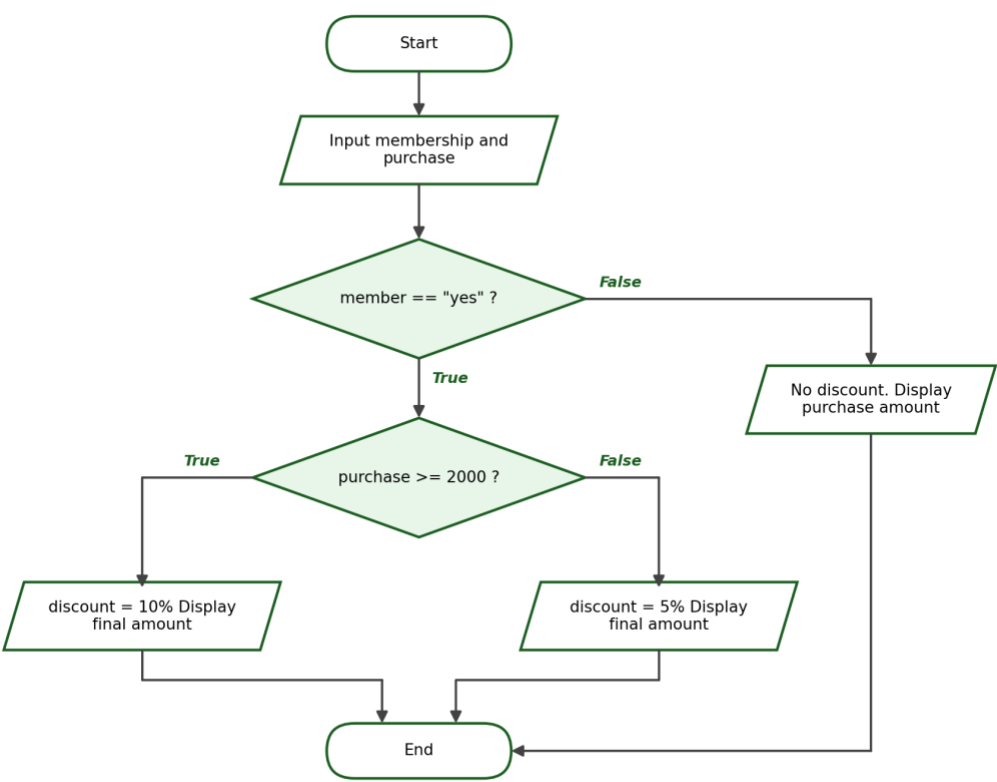
Algorithm:

- Step 1: Start.
- Step 2: Ask whether the customer is a member. Store the answer as member.
- Step 3: Ask the user to enter the purchase amount. Store it as purchase.
- Step 4: If member equals “yes”: if purchase >= 2000, compute discount = purchase × 0.10; otherwise compute discount = purchase × 0.05. Then compute final_amount = purchase – discount and display both.
- Step 5: If member does not equal “yes”, display “No discount.” and a final amount equal to the purchase.
- Step 6: End.

Pseudocode:

```
START
INPUT member
INPUT purchase
IF member == "yes" THEN
  IF purchase >= 2000 THEN
    discount = purchase * 0.10
  ELSE
    discount = purchase * 0.05
  END IF
  final_amount = purchase - discount
  DISPLAY discount, final_amount
ELSE
  DISPLAY "No discount."
  DISPLAY purchase
END IF
END
```

Flowchart:



Python Program:

```
member = input("Is the customer a member (yes/no)? ")
purchase = float(input("Enter purchase amount: "))
if member == "yes":
    if purchase >= 2000:
        discount = purchase * 0.10
    else:
        discount = purchase * 0.05
    final_amount = purchase - discount
    print(f"Discount: ₹{discount:.2f}")
    print(f"Final Amount: ₹{final_amount:.2f}")
else:
    print("No discount.")
    print(f"Final Amount: ₹{purchase:.2f}")
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	yes and 2500	Discount: ₱250.00 / Final Amount: ₱2250.00
2	no and 3000	No discount. / Final Amount: ₱3000.00
3	yes and 2000	Discount: ₱200.00 / Final Amount: ₱1800.00 (boundary)

Explanation: Verified arithmetic: 10% of ₱2,500 is ₱250 (final ₱2,250); the boundary purchase of exactly ₱2,000 earns the 10% rate (₱200 discount, final ₱1,800). The nesting stays one level deep as required.

9. Laboratory Activity 5 Answer Key — Complete Problem-Solving Tasks

Activity 5, Problem 1: Electricity-Bill Classification

IPO Table:

Input	Process	Output
kwh	Compare kwh against 100 and 300 from lowest to highest	Low usage / Moderate usage / High usage

Condition / Formula: kwh <= 100, kwh <= 300, else High usage

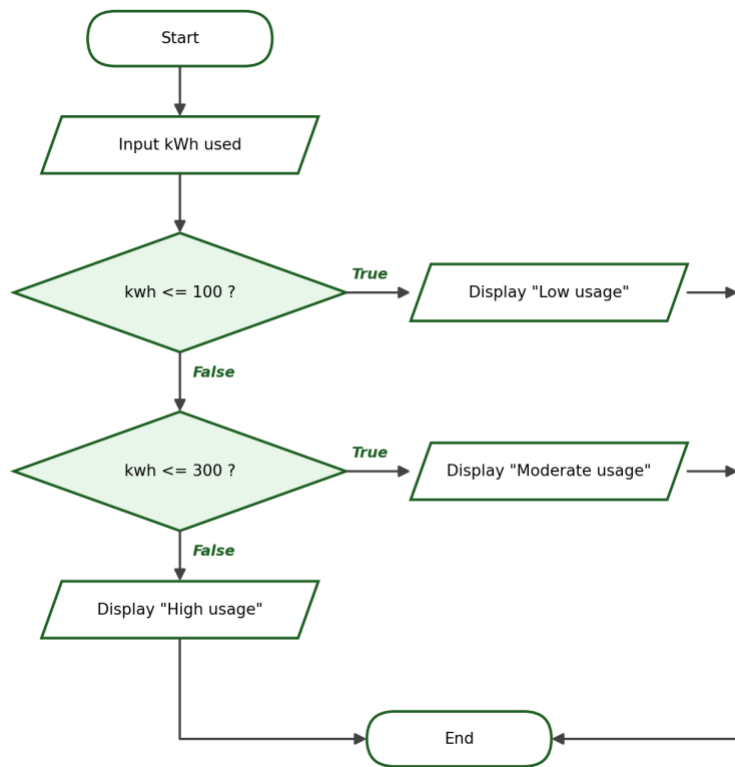
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the consumption in kWh. Store it as kwh.
- Step 3: If kwh <= 100, display “Low usage”.
- Step 4: Otherwise, if kwh <= 300, display “Moderate usage”.
- Step 5: Otherwise, display “High usage”.
- Step 6: End.

Pseudocode:

```
START
INPUT kwh
IF kwh <= 100 THEN
    DISPLAY "Low usage"
ELSE IF kwh <= 300 THEN
    DISPLAY "Moderate usage"
ELSE
    DISPLAY "High usage"
END IF
END
```

Flowchart:



Python Program:

```

kwh = float(input("Enter electricity consumption in kWh: "))
if kwh <= 100:
    print("Low usage")
elif kwh <= 300:
    print("Moderate usage")
else:
    print("High usage")

```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	80	Low usage
2	250	Moderate usage
3	100	Low usage (boundary)

Explanation: The boundary 100 belongs to “Low usage” because the range is “up to 100 kWh”. The <= conditions are ordered from lowest to highest boundary.

Activity 5, Problem 2: Parking Fee

IPO Table:

Input	Process	Output
hours	fee = 50 when hours <= 3; otherwise fee = 50 + (hours – 3) × 20	Parking fee

Condition / Formula: hours <= 3 → fee = 50; else fee = 50 + (hours – 3) × 20

Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the number of hours parked. Store it as hours.
- Step 3: If hours <= 3, set fee = 50.

Step 4: Otherwise, set fee = 50 + (hours – 3) × 20.

Step 5: Display the parking fee.

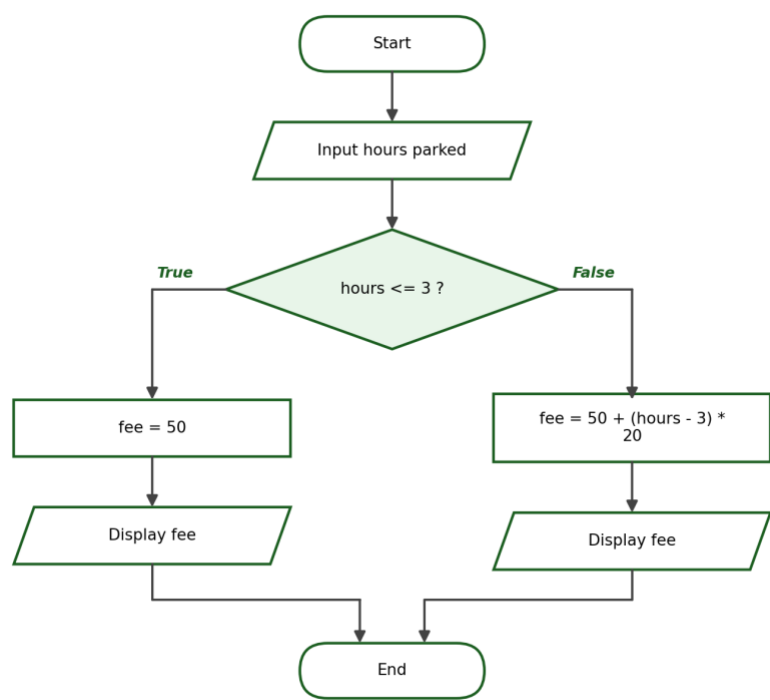
Step 6: End.

Pseudocode:

```

START
INPUT hours
IF hours <= 3 THEN
    fee = 50
ELSE
    fee = 50 + (hours - 3) * 20
END IF
DISPLAY "Parking fee: " + fee
END
    
```

Flowchart:



Python Program:

```

hours = int(input("Enter number of hours parked: "))
if hours <= 3:
    fee = 50
else:
    fee = 50 + (hours - 3) * 20
print(f"Parking fee: ₱{fee:.2f}")
    
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	2	Parking fee: ₱50.00
2	5	Parking fee: ₱90.00
3	3	Parking fee: ₱50.00 (boundary)

Explanation: Verified arithmetic for 5 hours: ₱50 + (5 – 3) × ₱20 = ₱90. Exactly 3 hours still costs the flat ₱50, which the boundary test confirms.

Activity 5, Problem 3: Delivery Charge

IPO Table:

Input	Process	Output
distance	Compare distance against 3 and 10 from lowest to highest; select the charge	Delivery charge (P40 / P70 / P120)

Condition / Formula: distance <= 3 → 40; distance <= 10 → 70; else 120

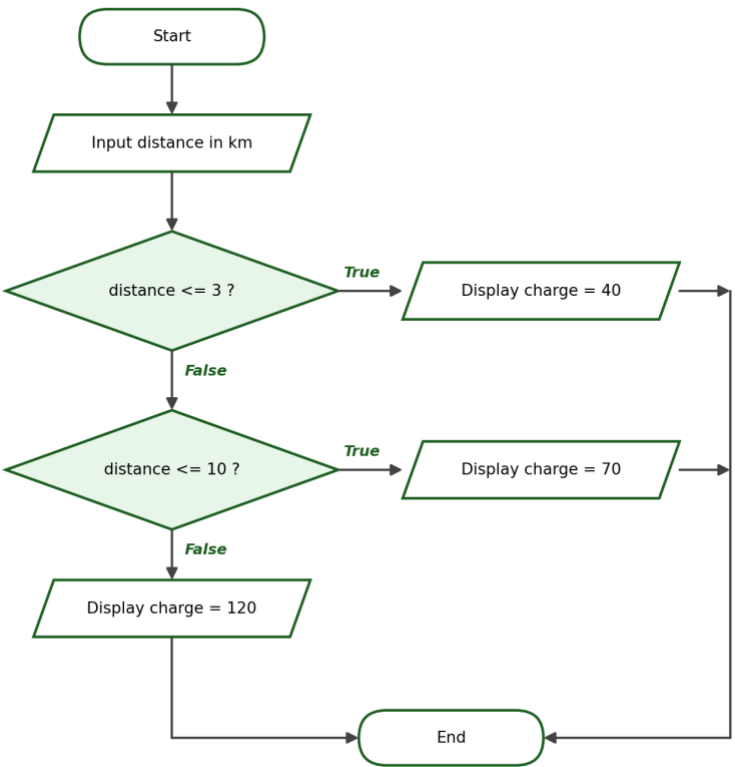
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the distance in kilometers. Store it as distance.
- Step 3: If distance <= 3, set charge = 40.
- Step 4: Otherwise, if distance <= 10, set charge = 70.
- Step 5: Otherwise, set charge = 120.
- Step 6: Display the delivery charge.
- Step 7: End.

Pseudocode:

```
START
INPUT distance
IF distance <= 3 THEN
    charge = 40
ELSE IF distance <= 10 THEN
    charge = 70
ELSE
    charge = 120
END IF
DISPLAY "Delivery charge: " + charge
END
```

Flowchart:



Python Program:

```
distance = float(input("Enter delivery distance in km: "))
if distance <= 3:
    charge = 40
elif distance <= 10:
    charge = 70
else:
    charge = 120
print(f"Delivery charge: ₱{charge:.2f}")
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	2	Delivery charge: ₱40.00
2	8	Delivery charge: ₱70.00
3	3	Delivery charge: ₱40.00 (boundary)

Explanation: The boundary 3 km still belongs to the ₱40 range. Writing < instead of <= would wrongly charge ₱70 for exactly 3 km — the boundary test catches this.

Activity 5, Problem 4: Ticket Price

IPO Table:

Input	Process	Output
age	Compare age against 4, 12, 59 from lowest to highest; select the price	Free admission, or the ticket price

Condition / Formula: age <= 4 → free; age <= 12 → 150; age <= 59 → 300; else 240

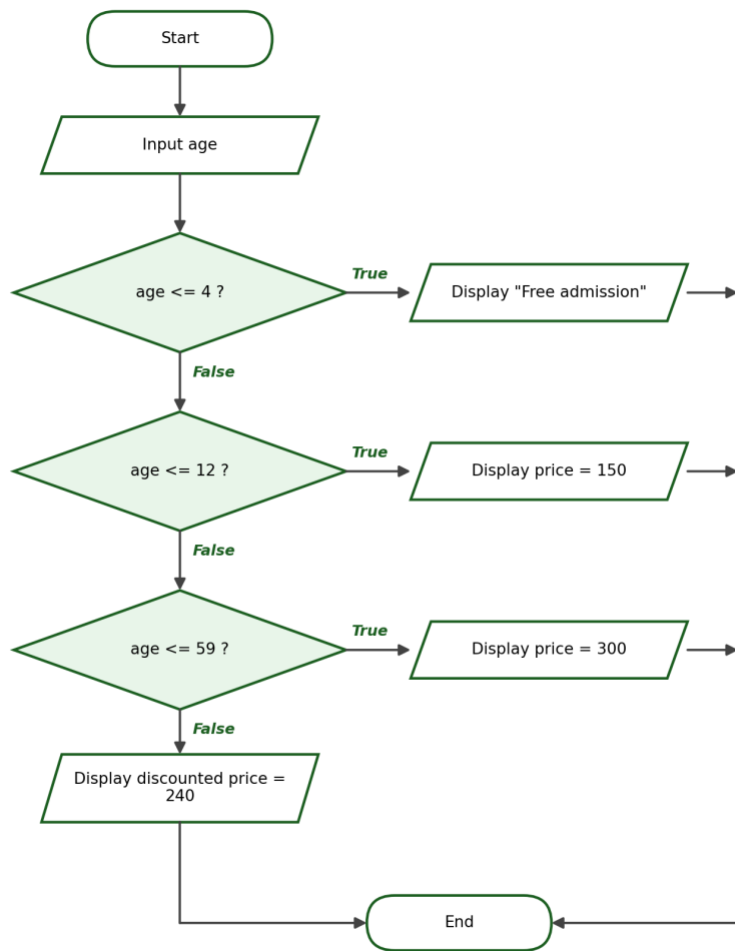
Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the age. Store it as age.
- Step 3: If age <= 4, display “Free admission”.
- Step 4: Otherwise, if age <= 12, display the price ₱150.
- Step 5: Otherwise, if age <= 59, display the price ₱300.
- Step 6: Otherwise, display the discounted price ₱240.
- Step 7: End.

Pseudocode:

```
START
INPUT age
IF age <= 4 THEN
    DISPLAY "Free admission"
ELSE IF age <= 12 THEN
    DISPLAY "Ticket price: 150"
ELSE IF age <= 59 THEN
    DISPLAY "Ticket price: 300"
ELSE
    DISPLAY "Ticket price: 240"
END IF
END
```

Flowchart:



Python Program:

```

age = int(input("Enter the age: "))
if age <= 4:
    print("Free admission")
elif age <= 12:
    price = 150
    print(f"Ticket price: ₱{price:.2f}")
elif age <= 59:
    price = 300
    print(f"Ticket price: ₱{price:.2f}")
else:
    price = 240
    print(f"Ticket price: ₱{price:.2f}")

```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	3	Free admission
2	30	Ticket price: ₱300.00
3	60	Ticket price: ₱240.00 (boundary)

Explanation: The four age categories are covered by three <= conditions plus an else. The boundary test confirms that 60 is the first age to receive the senior discounted price.

Activity 5, Problem 5: Largest of Three Numbers

IPO Table:

Input	Process	Output
num1, num2, num3	Compare the numbers using and; store the largest	The largest number

Condition / Formula: num1 >= num2 and num1 >= num3; otherwise num2 >= num3; otherwise num3

Algorithm:

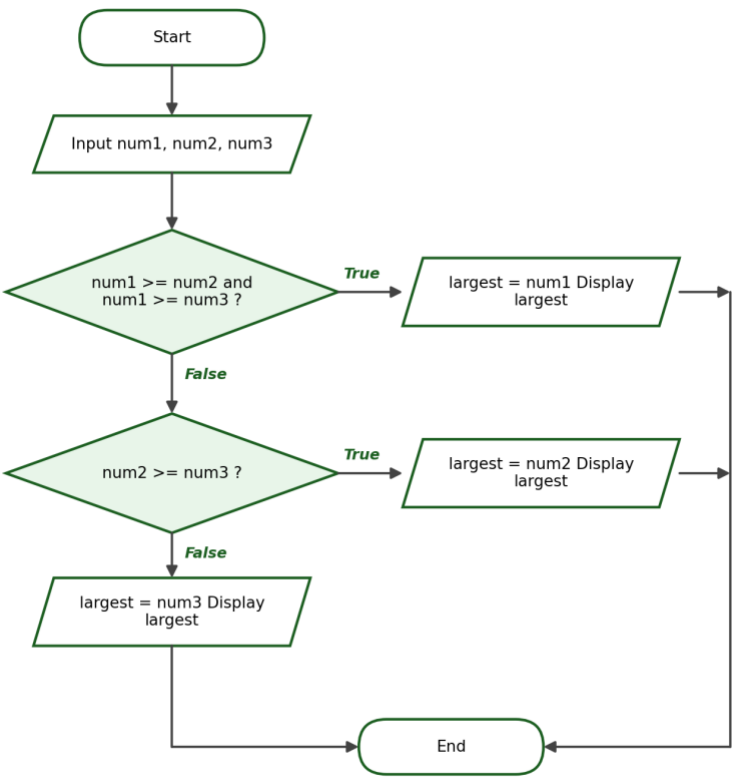
- Step 1: Start.
- Step 2: Ask the user to enter three numbers. Store them as num1, num2, and num3.
- Step 3: If num1 >= num2 and num1 >= num3, set largest = num1.
- Step 4: Otherwise, if num2 >= num3, set largest = num2.
- Step 5: Otherwise, set largest = num3.
- Step 6: Display the largest number.
- Step 7: End.

Pseudocode:

```

START
INPUT num1, num2, num3
IF num1 >= num2 AND num1 >= num3 THEN
    largest = num1
ELSE IF num2 >= num3 THEN
    largest = num2
ELSE
    largest = num3
END IF
DISPLAY "The largest number is " + largest
END
  
```

Flowchart:



Python Program:

```

num1 = float(input("Enter the first number: "))
num2 = float(input("Enter the second number: "))
num3 = float(input("Enter the third number: "))
if num1 >= num2 and num1 >= num3:
    largest = num1
elif num2 >= num3:
    largest = num2
else:
    largest = num3
print(f"The largest number is {largest}")

```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	5, 9, 3	The largest number is 9.0
2	10, 2, 7	The largest number is 10.0
3	4, 4, 4	The largest number is 4.0 (boundary: all equal)

Explanation: Using >= (not >) makes the program correct even when values are tied: with all inputs equal, the first condition is True and num1 is reported, which is a valid largest value.

Activity 5, Problem 6: Employee Overtime Pay

IPO Table:

Input	Process	Output
hours, rate	If hours > 40: pay = 40 × rate + (hours – 40) × rate × 1.25; else pay = hours × rate	Classification and total pay

Condition / Formula: hours > 40; overtime portion paid at 1.25 × rate

Algorithm:

- Step 1: Start.
- Step 2: Ask the user to enter the hours worked. Store it as hours.
- Step 3: Ask the user to enter the hourly rate. Store it as rate.
- Step 4: If hours > 40, compute regular_pay = 40 × rate, overtime_pay = (hours – 40) × rate × 1.25, and total_pay = regular_pay + overtime_pay, then display “With overtime” and the total pay.
- Step 5: Otherwise, compute total_pay = hours × rate and display “Regular” and the total pay.
- Step 6: End.

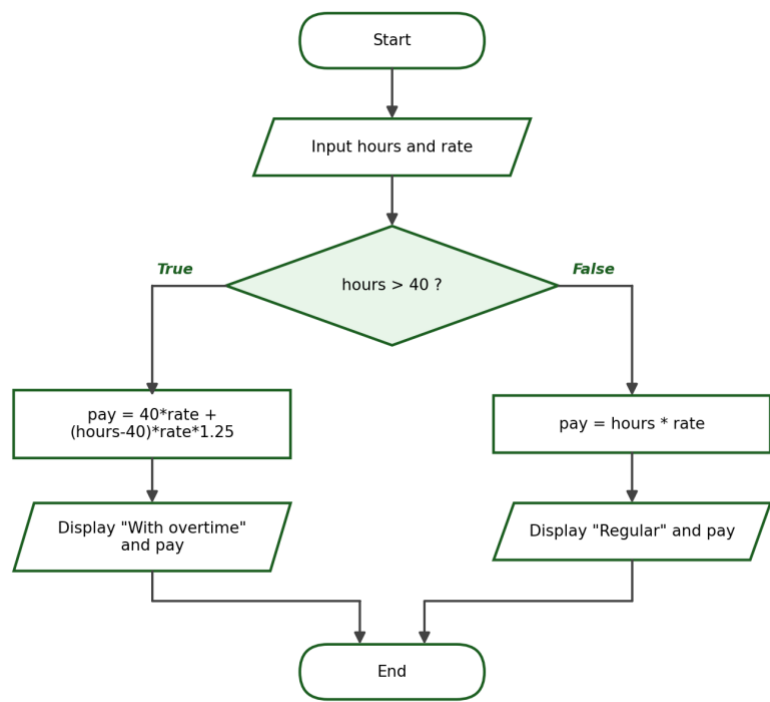
Pseudocode:

```

START
INPUT hours
INPUT rate
IF hours > 40 THEN
    regular_pay = 40 * rate
    overtime_pay = (hours - 40) * rate * 1.25
    total_pay = regular_pay + overtime_pay
    DISPLAY "With overtime", total_pay
ELSE
    total_pay = hours * rate
    DISPLAY "Regular", total_pay
END IF
END

```

Flowchart:



Python Program:

```
hours = float(input("Enter hours worked this week: "))
rate = float(input("Enter hourly rate: "))
if hours > 40:
    regular_pay = 40 * rate
    overtime_pay = (hours - 40) * rate * 1.25
    total_pay = regular_pay + overtime_pay
    print("Classification: With overtime")
    print(f"Total Pay: ₱{total_pay:.2f}")
else:
    total_pay = hours * rate
    print("Classification: Regular")
    print(f"Total Pay: ₱{total_pay:.2f}")
```

Test Cases and Verified Outputs:

Test Case	Input	Expected / Verified Output
1	45 hours at ₱100	Classification: With overtime / Total Pay: ₱4625.00
2	38 hours at ₱100	Classification: Regular / Total Pay: ₱3800.00
3	40 hours at ₱100	Classification: Regular / Total Pay: ₱4000.00 (boundary)

Explanation: Verified arithmetic for 45 hours at ₱100: $40 \times 100 = ₱4,000$ regular plus $5 \times 100 \times 1.25 = ₱625$ overtime, totaling ₱4,625. Exactly 40 hours is Regular pay, which the boundary test confirms.

10. Grading Reminders

Accept alternative solutions that produce the required outputs with correct logic, meaningful variable names, and proper indentation. Deduct points for hard-coded outputs (printing answers without evaluating conditions), missing type conversion, incorrect boundary behavior, and range gaps or overlaps. Cross-check each student’s GitHub commit history: the work should have been committed from the local machine, not edited through the GitHub website.